



Project 3.1: Heart Disease Diagnosis

Dương Đình Thắng

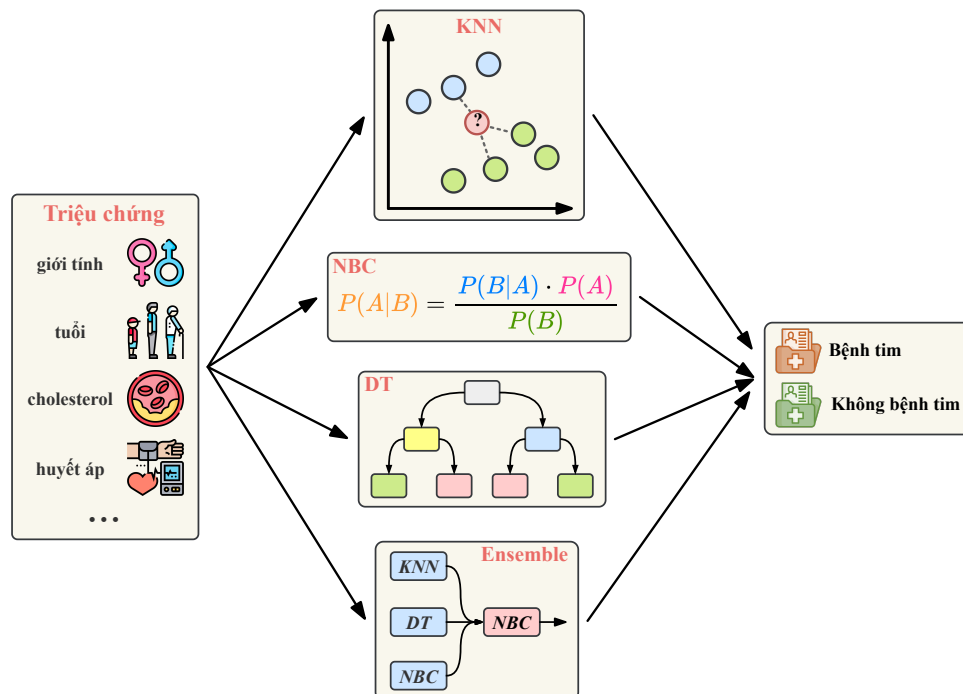
Nguyễn Anh Khôi

Đình Quang Vinh

I. Dẫn nhập

Bệnh tim mạch hiện là nguyên nhân gây tử vong hàng đầu trên thế giới, với khoảng 20,5 triệu ca vào năm 2023, chiếm xấp xỉ 32% tổng số ca tử vong toàn cầu [1], [2], [3]. Tại Việt Nam, ước tính của WHO năm 2016 cho thấy 31% trường hợp tử vong có liên quan đến bệnh tim mạch. Dự báo mới nhất cũng chỉ ra rằng 80% gánh nặng tử vong do bệnh tim sẽ rơi vào các quốc gia có thu nhập từ thấp đến trung bình - nơi việc tầm soát và điều trị còn gặp nhiều hạn chế về chi phí và nguồn lực [3], [4].

Trước thực trạng đó, các giải pháp hỗ trợ chẩn đoán sớm và chính xác đóng vai trò vô cùng quan trọng. Trong những năm gần đây, các mô hình học máy (machine learning) đã nổi lên như một hướng tiếp cận hiệu quả, nhờ khả năng xử lý khối lượng dữ liệu lớn và phát hiện các mối quan hệ ẩn giữa các đặc trưng y tế. Đặc biệt, nhiều nghiên cứu đã chứng minh rằng các thuật toán học máy truyền thống như **Naive Bayes**, **K-Nearest Neighbors**, **Decision Tree**... vẫn được ưa chuộng nhờ tính đơn giản, khả năng diễn giải, và độ chính xác chấp nhận được trong thực tế.

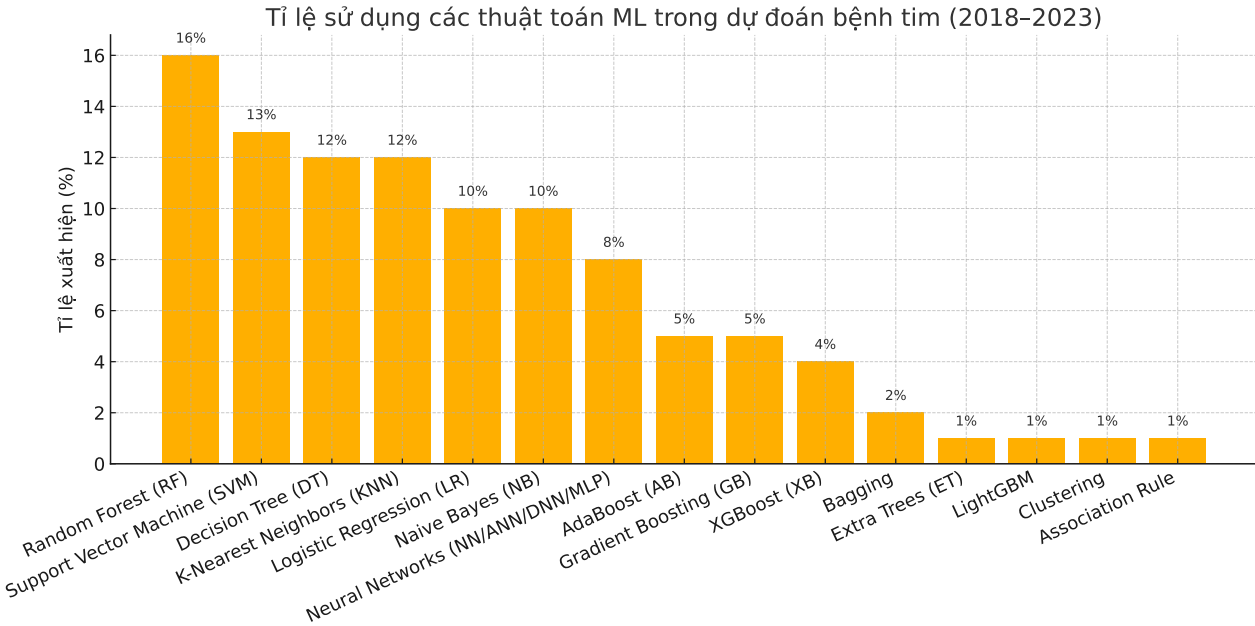


Hình 1: Tổng quát dự án dự đoán bệnh tim bằng các phương pháp học máy.

Mục lục

I.	Dẫn nhập	1
II.	Cài đặt chương trình	4
II.1.	Tổng quan về bộ dữ liệu	4
II.2.	Kỹ thuật xử lý đặc trưng	6
II.3.	Naive Bayes Classifier	14
II.4.	K-Nearest Neighbors	18
II.5.	Decision Tree	21
II.6.	K-means	23
II.7.	Cải tiến: Ensemble	25
III.	Câu hỏi trắc nghiệm	29
IV.	Tài liệu tham khảo	31
	Phụ lục	32

Theo một công trình khảo sát [5] (2025) về bài toán liên quan cung cấp góc nhìn về mức độ phổ biến của các kỹ thuật học máy truyền thống thường được sử dụng (trực quan như hình Hình 2 bên dưới). Một cách bất ngờ rằng các mô hình cơ bản như **K-Nearest Neighbors KNN**, **Decision Tree DT** với 12% và **Naive Bayes NB** là 10%, cho thấy các mô hình này vẫn được ưu chuộng vì tính đơn giản mà vẫn đem lại độ chính xác chấp nhận được.



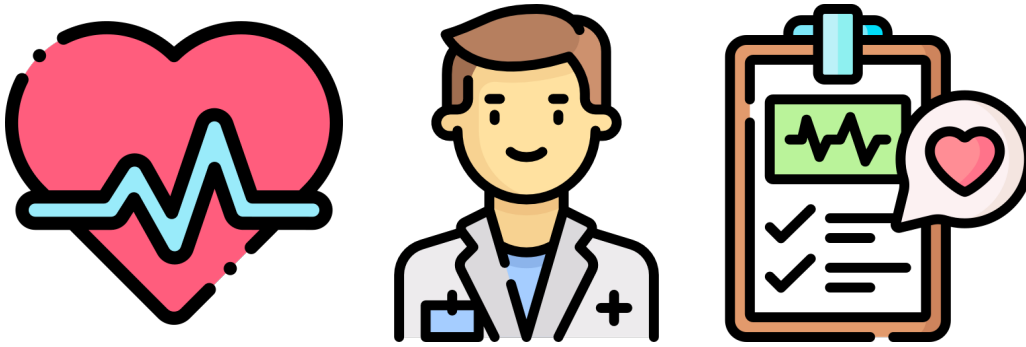
Hình 2: Trực quan tần suất xuất hiện của các mô hình học máy truyền thống trong các công trình nghiên cứu về dự đoán bệnh tim từ năm 2018 đến 2023.

Xuất phát từ bối cảnh này, tài liệu sẽ hướng dẫn cách xây dựng và so sánh nhiều mô hình học máy trên bộ dữ liệu bệnh tim. Nội dung tiếp theo bao gồm: Tổng quan dữ liệu, kỹ thuật xử lý đặc trưng, cùng phần giới thiệu và thực hành với các mô hình học máy trên hai phiên bản dữ liệu: bộ dữ liệu gốc và bộ dữ liệu đã được xử lý đặc trưng.

II. Cài đặt chương trình

II.1. Tổng quan về bộ dữ liệu

“Biết người biết ta, trăm trận trăm thắng”, trong học máy, việc hiểu rõ dữ liệu chính là yếu tố quyết định để xây dựng mô hình hiệu quả. Trong dự án này, chúng ta sử dụng bộ dữ liệu Cleveland Heart Disease, một trong bốn tập con thuộc bộ dữ liệu bệnh tim tổng hợp do UCI Machine Learning Repository cung cấp [6]. So với các tập con khác (Hungary, Switzerland, Long Beach VA), Cleveland được lựa chọn rộng rãi trong nghiên cứu nhờ có ít giá trị thiếu, nhãn phân loại rõ ràng và xuất phát từ nghiên cứu y học kinh điển của Detrano và cộng sự [7].



Cleveland Heart Disease Diagnosis

Hình 3: Bộ dữ liệu dự đoán bệnh tim: Cleveland Heart Disease Diagnosis.

Bộ dữ liệu gồm 303 bệnh nhân với 14 đặc trưng y tế phản ánh tình trạng sức khỏe tim mạch, bao gồm các thông tin nhân khẩu học (tuổi, giới tính), các chỉ số lâm sàng (huyết áp, cholesterol, nhịp tim tối đa), các kết quả kiểm tra chuyên sâu (điện tâm đồ, mức độ trầm ST, số mạch máu chính) và thông tin về bệnh lý thalassemia. Mỗi đặc trưng đã được mã hóa dạng số để thuận tiện cho việc huấn luyện mô hình. Bảng dưới đây minh họa một vài mẫu dữ liệu được lấy từ bộ dữ liệu gốc:

Bảng 1: Ví dụ một số mẫu dữ liệu từ bộ Cleveland Heart Disease.

age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0
67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	1
67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0
41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0

Việc nắm vững cấu trúc và ý nghĩa của các đặc trưng này không chỉ giúp lựa chọn mô hình phù hợp mà còn đóng vai trò quan trọng trong các bước xử lý đặc trưng và phân tích kết quả. Ở phần tiếp theo, chúng ta sẽ tìm hiểu các kỹ thuật xử lý đặc trưng cần thiết trước khi huấn luyện mô hình.

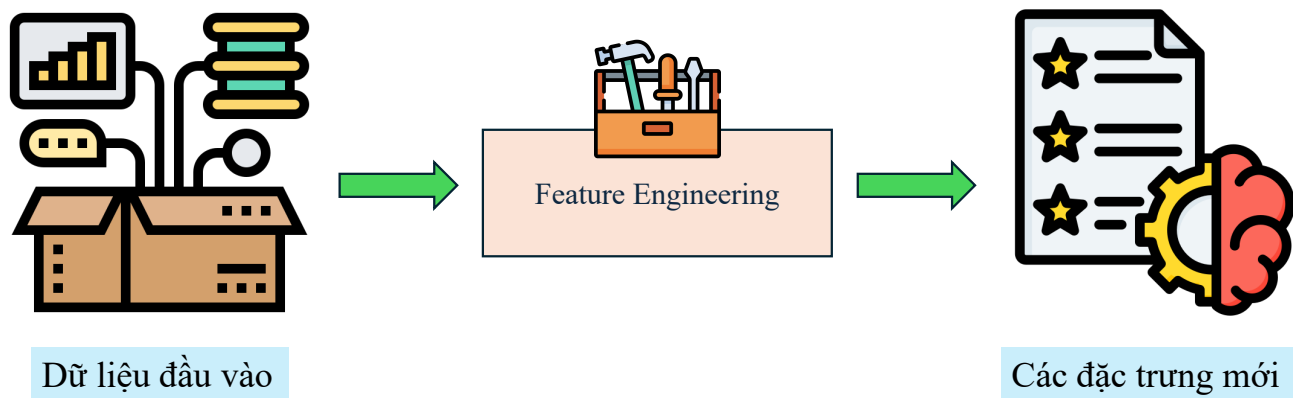
Bảng mô tả đặc trưng bộ dữ liệu Cleveland Heart Disease

Đặc trưng	Mô tả và giá trị mã hóa
age	Tuổi của bệnh nhân (năm).
sex	Giới tính. (1 = nam, 0 = nữ)
cp	Loại đau ngực. (1 = đau thắt ngực điển hình, 2 = không điển hình, 3 = đau không do tim, 4 = không triệu chứng)
trestbps	Huyết áp tâm thu lúc nghỉ (mmHg).
chol	Nồng độ cholesterol huyết thanh (mg/dL).
fbs	Đường huyết lúc đói > 120 mg/dL. (1 = đúng, 0 = sai)
restecg	Điện tâm đồ lúc nghỉ. (0 = bình thường, 1 = bất thường ST-T, 2 = phì đại thất trái)
thalach	Nhịp tim tối đa đạt được (lần/phút).
exang	Đau ngực khi gắng sức. (1 = có, 0 = không)
oldpeak	Mức độ trầm ST do gắng sức so với nghỉ.
slope	Độ dốc đoạn ST. (1 = dốc lên, 2 = bằng phẳng, 3 = dốc xuống)
ca	Số mạch máu chính được nhuộm màu (0–3).
thal	Thalassemia. (3 = bình thường, 6 = tổn thương cố định, 7 = tổn thương có thể đảo ngược)
num	Nhãn mục tiêu. (0 = không bệnh, 1–4 = có bệnh)

Các giá trị rời rạc được mã hóa trong ngoặc để phục vụ huấn luyện mô hình.

II.2. Kỹ thuật xử lý đặc trưng

Feature engineering là một kỹ thuật đặc biệt có thể biến đổi và bổ sung đặc trưng từ dữ liệu gốc để mô hình có khả năng học tốt hơn. Việc này giúp làm nổi bật các tín hiệu hữu ích, giảm nhiễu, đồng thời tạo ra các đặc trưng mới có giá trị phân tách cao hơn cho quá trình huấn luyện. Trong bài toán dự đoán bệnh tim, chúng ta có thể tiến hành xử lý đặc trưng nhằm cải thiện chất lượng dữ liệu đầu vào, từ đó nâng cao hiệu quả của các mô hình học máy. Việc so sánh kết quả trên tập dữ liệu gốc và tập đã xử lý đặc trưng sẽ cho thấy mức độ cải thiện rõ rệt của kỹ thuật này. **Lưu ý:** Các bạn có thể tải trước bộ dữ liệu sau khi feature engineering tại phần IV.



Hình 4: Minh họa về kỹ thuật feature engineering khi áp dụng vào một bộ dữ liệu nhằm tạo thêm các đặc trưng mới tốt hơn.

Trước tiên, nạp các thư viện xử lý dữ liệu, tiền xử lý, chọn đặc trưng và trực quan, ngoài ra đặt seed để kết quả có thể lặp lại ổn định.

```

1 import os
2 import json
3 import random
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7
8 from pathlib import Path
9 from sklearn.pipeline import Pipeline
10 from sklearn.impute import SimpleImputer
11 from sklearn.compose import ColumnTransformer
12 from sklearn.linear_model import LinearRegression
13 from sklearn.model_selection import train_test_split
14 from sklearn.feature_selection import VarianceThreshold, mutual_info_classif
15 from sklearn.preprocessing import (
16     OneHotEncoder, StandardScaler,
17     FunctionTransformer, MinMaxScaler
18 )
19
20 os.environ["PYTHONHASHSEED"] = "42"
21 np.random.seed(42)
  
```

```

22 random.seed(42)
23 print("Seed: 42")

```

Tiếp theo là tải tập dữ liệu Cleveland từ Google Drive bằng `gdown` để chuẩn bị cho các bước làm sạch và tách thành các tập con, các bạn có thể kiểm tra bộ dữ liệu tại phần IV.

```

1 !gdown 16HPyuXWPptt5g3xvS_kR_wXAfjpR1Ju

```

Kế tiếp, đặt tên cột theo các kiểu chuẩn, ép kiểu số cho các trường quan trọng, nhị phân hóa cho cột `target` (0: không bệnh và > 1: có bệnh).

```

1 DATA_PATH = "cleveland.csv"
2 COLUMNS = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
3             "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target"]
4
5 raw = pd.read_csv(DATA_PATH, header=None)
6 raw.columns = COLUMNS
7
8 for c in ["age", "trestbps", "chol", "thalach", "oldpeak", "ca", "thal"]:
9     raw[c] = pd.to_numeric(raw[c], errors="coerce")
10
11 raw["target"] = (raw["target"] > 0).astype(int)
12 print("Shape:", raw.shape)
13 display(raw.head())
14 display(raw.isna().sum())

```

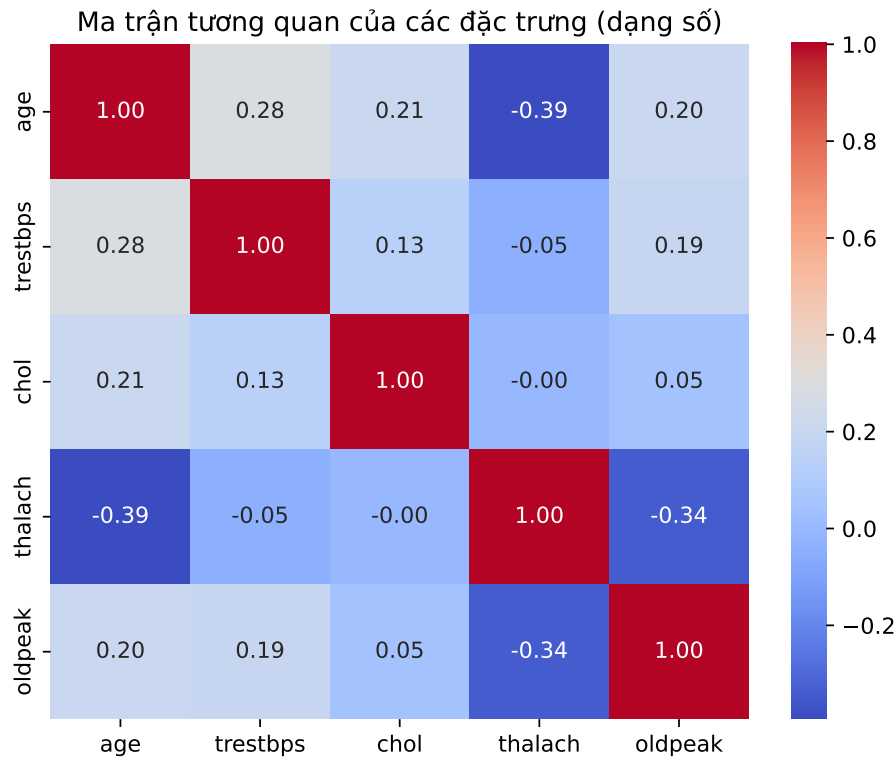
Sau đó, tách danh sách cột dạng “số” và cột dạng “phân loại”, làm cơ sở cho 2 luồng tiền xử lý khác nhau.

```

1 numeric_cols = ["age", "trestbps", "chol", "thalach", "oldpeak"]
2 categorical_cols = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]

```

Tiếp đến là khám phá dữ liệu như trực quan ma trận tương quan, phân phối của các đặc trưng dạng số.



Hình 5: Ma trận tương quan giữa các đặc trưng.

```

1 plt.figure(figsize=(6, 5))
2 corr_matrix = raw[numeric_cols].corr()
3 sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
4 plt.title("Ma trận tương quan của các đặc trưng (dạng số)")
5 plt.tight_layout()
6 plt.show()
7
8 sns.pairplot(raw, vars=numeric_cols, hue="target", diag_kind="kde")
9 plt.figure(figsize=(10, 10))
10 plt.suptitle("Phân phối theo từng cặp đặc trưng (dạng số) theo nhãn", y=1.02)
11 plt.savefig("pairwise_distribution.pdf", bbox_inches="tight")
12 plt.show()

```

Bên cạnh đó, tiến hành trực quan một số mối liên hệ như scatter age với các chỉ số (kèm đường hồi quy), xu hướng theo nhóm tuổi, và tương quan so với age, giúp gợi ý đặc trưng tiềm năng trước khi thực hiện tạo đặc trưng mới.



Hình 6: Mối quan hệ giữa tuổi và huyết áp tâm thu lúc nghỉ (trestbps), kèm đường hồi quy tuyến tính.

```

1 X, y = raw.drop(columns=["target"], raw["target"]
2 colors = np.where(y==0, "tab:blue", "tab:red")
3 metrics = [c for c in ["chol","trestbps","thalach"] if c in X.columns]
4
5 def scatter_with_regression(xs, ys, xlab, ylab, title):
6     m = xs.notna() & ys.notna()
7     x, yy = xs[m].values, ys[m].values
8     plt.figure()
9     plt.scatter(xs, ys, c=colors, alpha=0.7)
10    if len(x) > 1:
11        k, b = np.polyfit(x, yy, 1)
12        xline = np.linspace(np.nanmin(x), np.nanmax(x), 100)
13        plt.plot(xline, k*xline + b)
14    plt.xlabel(xlab)
15    plt.ylabel(ylab)
16    plt.title(title)
17    plt.tight_layout()
18    plt.show()
19
20 if "age" in X.columns and metrics:
21     for col in metrics:
22         scatter_with_regression(
23             X["age"], X[col],
24             "Age", col,
25             f"{col} vs Age (+ regression)")

```

```

26
27 if "age" in X.columns and metrics:
28     age_bins = pd.cut(X["age"], bins=5)
29     for m in metrics:
30         plt.figure()
31         avg_vals = X.groupby(age_bins, observed=True)[m].mean()
32         plt.plot(range(len(avg_vals)), avg_vals, marker="o", label=f"Average {m}")
33         plt.xticks(
34             range(len(avg_vals)), map(str, avg_vals.index),
35             rotation=45, ha="right"
36         )
37         plt.xlabel("Age group")
38         plt.ylabel(m)
39         plt.title(f"{m} average per age group")
40         plt.legend()
41         plt.tight_layout()
42         plt.show()
43
44 cols = [c for c in ["age", "chol", "trestbps", "thalach"] if c in X.columns]
45 if "age" in cols:
46     corr = X[cols].corr(numeric_only=True)["age"].drop("age", errors="ignore")
47     plt.figure(); corr.plot(kind="bar")
48     plt.ylabel("Correlation with age")
49     plt.title("Correlation of features with Age")
50     plt.tight_layout()
51     plt.show()

```

Chia bộ dữ liệu gốc thành 3 tập con train, val, test (tỉ lệ 80:10:10), sau đó biến đổi theo luồng xử lý như sau:

- Nếu cột là dạng “số”: thực hiện thay thế các giá trị thiếu bằng giá trị trung vị (median) trong cột đó, sau đó chuẩn hóa dữ liệu số về phân phối có trung bình là 0 và độ lệch chuẩn bằng 1.
- Nếu cột là dạng “phân loại”: thay thế giá trị thiếu bằng giá trị xuất hiện nhiều nhất (mode) trong cột đó, sau đó chuẩn hóa dữ liệu theo phương pháp Min-Max.

Sau đó lưu thành các tập gốc gồm raw_train/val/test.csv.

```

1 TARGET = "target"
2 feat_cols = raw.columns.drop(TARGET)
3 X_all, y_all = raw[feat_cols], raw[TARGET]
4 K = len(feat_cols)
5
6 num_proc = Pipeline([("imp", SimpleImputer(strategy="median")),
7                       ("sc", StandardScaler())])
8 cat_proc = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
9                       ("sc", MinMaxScaler())])
10
11 preprocess = ColumnTransformer([
12     ("num", num_proc, numeric_cols),
13     ("cat", cat_proc, categorical_cols),

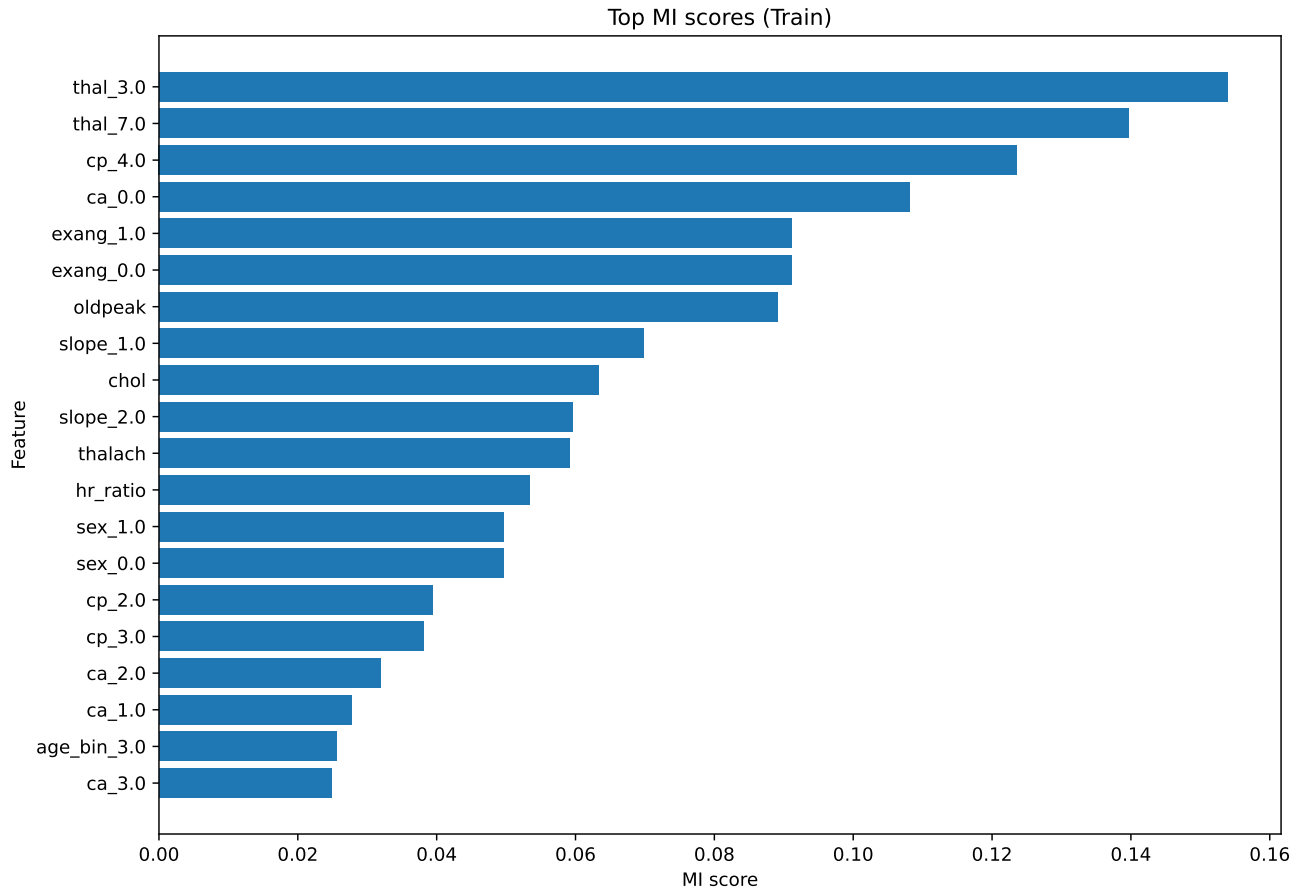
```

```

14 ], verbose_feature_names_out=False).set_output(transform="pandas")
15
16 raw_pipeline = Pipeline([("preprocess", preprocess)])
17
18 X_train, X_temp, y_train, y_temp = train_test_split(
19     X_all, y_all, test_size=0.2, stratify=y_all, random_state=42
20 )
21 X_val, X_test, y_val, y_test = train_test_split(
22     X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42
23 )
24
25 X_raw_train = raw_pipeline.fit_transform(X_train, y_train)
26 X_raw_val = raw_pipeline.transform(X_val)
27 X_raw_test = raw_pipeline.transform(X_test)
28
29 out_dir = Path("splits"); out_dir.mkdir(parents=True, exist_ok=True)
30 for name, (X_, y_) in {
31     "raw_train": (X_raw_train, y_train),
32     "raw_val": (X_raw_val, y_val),
33     "raw_test": (X_raw_test, y_test),
34 }.items():
35     pd.concat(
36         [X_, y_.rename(TARGET)], axis=1
37     ).to_csv(out_dir / f"{name}.csv", index=False)
38
39 print(f"Saved RAW splits. K (RAW features) = {K}")
40 display(X_raw_train)
41 display(X_raw_train.isna().sum())

```

Cuối cùng, tạo bộ dữ liệu được xử lý đặc trưng bằng cách sinh thêm đặc trưng mới theo độ tuổi. Sau đó dữ liệu được tiền xử lý theo hai luồng tương tự như trên tập gốc, nhưng các giá trị trong cột dạng “phân loại” được mã hóa theo One-hot. Sau đó, ước lượng mutual information (IM) để xếp hạng và chọn Top-K đặc trưng (với K bằng số lượng đặc trưng trên bộ gốc) và lưu thành các tập `fe_train/val/test.csv`.



Hình 7: Các đặc trưng quan trọng theo thang điểm Mutual Information (MI) trên tập huấn luyện.

```

1 def add_new_features(df):
2     df = df.copy()
3     if {"chol", "age"} <= set(df.columns):
4         df["chol_per_age"] = df["chol"]/df["age"]
5     if {"trestbps", "age"} <= set(df.columns):
6         df["bps_per_age"] = df["trestbps"]/df["age"]
7     if {"thalach", "age"} <= set(df.columns):
8         df["hr_ratio"] = df["thalach"]/df["age"]
9     if "age" in df.columns:
10        df["age_bin"] = pd.cut(df["age"], bins=5, labels=False).astype("category")
11    return df
12
13 gen_num = ["chol_per_age", "bps_per_age", "hr_ratio"]
14 gen_cat = ["age_bin"]
15 all_nums = [c for c in numeric_cols] + gen_num
16 all_cats = [c for c in categorical_cols] + gen_cat
17
18 num_proc = Pipeline([("imp", SimpleImputer(strategy="median")),
19                      ("sc", StandardScaler())])
20 cat_proc = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
21                      ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))])

```

```

    ]]])
22
23 pre = ColumnTransformer([
24     ("num", num_proc, all_nums),
25     ("cat", cat_proc, all_cats),
26 ], verbose_feature_names_out=False).set_output(transform="pandas")
27
28 fe_pre = Pipeline([
29     ("add", FunctionTransformer(add_new_features, validate=False)),
30     ("pre", pre),
31 ]).set_output(transform="pandas")
32
33 Xt_tr = fe_pre.fit_transform(X_train, y_train)
34 Xt_va = fe_pre.transform(X_val)
35 Xt_te = fe_pre.transform(X_test)
36
37 nz_cols = Xt_tr.columns[Xt_tr.nunique(dropna=False) > 1]
38 Xt_tr, Xt_va, Xt_te = Xt_tr[nz_cols], Xt_va[nz_cols], Xt_te[nz_cols]
39
40 ohe = fe_pre.named_steps["pre"].named_transformers_["cat"].named_steps["ohe"]
41 cat_names = list(ohe.get_feature_names_out(all_cats))
42 is_discrete = np.array([c in cat_names for c in Xt_tr.columns], dtype=bool)
43
44 mi = mutual_info_classif(Xt_tr.values, y_train.values,
45                          discrete_features=is_discrete, random_state=42)
46 mi_series = pd.Series(mi, index=Xt_tr.columns).sort_values(ascending=False)
47
48 K = raw.columns.drop("target").shape[0]
49 topk_cols = list(mi_series.head(K).index)
50
51 fe_tr = Xt_tr[topk_cols].assign(target=y_train.values)
52 fe_va = Xt_va[topk_cols].assign(target=y_val.values)
53 fe_te = Xt_te[topk_cols].assign(target=y_test.values)
54
55 out = Path("splits"); out.mkdir(parents=True, exist_ok=True)
56 fe_tr.to_csv(out/"fe_train.csv", index=False)
57 fe_va.to_csv(out/"fe_val.csv", index=False)
58 fe_te.to_csv(out/"fe_test.csv", index=False)
59
60 print(f"Saved FE splits. K (FE features) = {K}")
61 display(pd.Series(topk_cols, name="fe_topk_features").reset_index(drop=True))
62
63 N = min(20, len(mi_series))
64 topN = mi_series.head(N).iloc[::-1]
65 plt.figure(figsize=(10, max(6, 0.35*N)))
66 plt.barh(topN.index, topN.values)
67 plt.title("Top MI scores (Train)")
68 plt.xlabel("MI score")
69 plt.ylabel("Feature")
70 plt.tight_layout()
71 plt.show()

```

Sau khi xử lý bộ dữ liệu với các bước trên, chạy code bên dưới để nén folder thành file ZIP

database.zip, trong đó gồm:

- Bộ dữ liệu gốc: 3 file fe_train/val/test.csv.
- Bộ dữ liệu bộ dữ liệu được xử lý đặc trưng: 3 file fe_train/val/test.csv.

```
1 !zip -r dataset.zip splits
2
3 # adding: splits/
4 # adding: splits/fe_train.csv
5 # adding: splits/raw_test.csv
6 # adding: splits/raw_val.csv
7 # adding: splits/fe_test.csv
8 # adding: splits/fe_val.csv
9 # adding: splits/raw_train.csv
```

II.3. Naive Bayes Classifier

II.3.1. Giới thiệu

Bắt đầu với Naive Bayes Classifier, một nhóm mô hình phân loại dựa trên định lý Bayes, tính toán xác suất hậu nghiệm của mỗi lớp và gán mẫu dữ liệu vào lớp có xác suất cao nhất.

$$\begin{array}{c}
 \text{Khả năng xảy ra} \quad \text{Xác suất tiên nghiệm} \\
 P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \\
 \text{Xác suất hậu nghiệm} \qquad \qquad \text{Bằng chứng}
 \end{array}$$

Hình 8: Công thức Naive Bayes và các thành phần liên quan.

Để đơn giản hóa quá trình tính toán các xác suất phức tạp, mô hình áp dụng giả định “ngây thơ” (naive) rằng các đặc trưng đầu vào độc lập có điều kiện khi đã biết nhãn lớp. Nói cách khác, một khi biết bệnh nhân thuộc lớp “mắc bệnh tim” hay “không mắc bệnh tim”, giá trị của một đặc trưng (ví dụ: cholesterol) sẽ không ảnh hưởng đến giá trị của đặc trưng khác (ví dụ: huyết áp) trong việc ước lượng xác suất thuộc lớp. Mặc dù giả định đơn giản, Naive Bayes vẫn cho thấy hiệu quả tốt trong nhiều trường hợp, đặc biệt khi dữ liệu có số lượng mẫu hạn chế và đặc trưng rời rạc.

II.3.2. Triển khai

Lưu ý: Các khối code được đánh dấu “**Tái sử dụng**” nghĩa là được sử dụng nhiều lần mà không cần sửa đổi giữa các mô hình khác nhau trong dự án này.

Bắt đầu bằng việc nạp các thư viện cần thiết và đặt seed để giữ kết quả nhất quán giữa các lần chạy.

Tái sử dụng

```

1  # Import thư viện cần thiết
2  import os
3  import random
4  import warnings
5  import numpy as np
6  import pandas as pd
7  import seaborn as sns
8  import matplotlib.pyplot as plt
9
10 from collections import Counter
11 from sklearn.cluster import KMeans
12 from sklearn.pipeline import Pipeline
13 from sklearn.naive_bayes import GaussianNB
14 from sklearn.ensemble import StackingClassifier
15 from sklearn.tree import DecisionTreeClassifier
16 from sklearn.neighbors import KNeighborsClassifier
17 from sklearn.preprocessing import StandardScaler
18 from sklearn.metrics import accuracy_score, classification_report
19 from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
20
21 warnings.filterwarnings("ignore")
22
23 # Đặt seed để đảm bảo kết quả nhất quán giữa các lần chạy
24 SEED = 42
25 os.environ["PYTHONHASHSEED"] = str(SEED)
26 np.random.seed(SEED)
27 random.seed(SEED)
28 print(f"Seed: {SEED}")

```

Tiếp theo, tải bộ dữ liệu đã được xử lý như ở mục II.2., khi này bộ dữ liệu gồm 2 loại: bộ gốc và bộ được xử lý đặc trưng (được đánh dấu bằng ký hiệu “fe”), mỗi bộ gồm các tập train, val, test đã được chia sẵn.

Tái sử dụng

```

1  !gdown 1T6AWCoyeC2MqGvmqZPPcA4ni5Md1k0sI
2  !unzip dataset.zip

```

Kế tiếp, xây dựng một hàm đọc file CSV, tách các đặc trưng X và nhãn y để phục vụ cho việc huấn luyện mô hình, ngoài ra, hiển thị thêm một số thông tin cơ bản của tập dữ liệu.

Tái sử dụng

```

1  def read_csv(file_path):
2      df = pd.read_csv(file_path)

```

```

3     display(df.head())
4     X = df.drop("target", axis=1)
5     y = df["target"]
6
7     display(y.value_counts())
8     print("Shape df: ", df.shape)
9     print("Shape X: ", X.shape)
10    print("Shape y: ", y.shape)
11    return X, y

```

Sau đó, lần lượt đọc các tập dữ liệu gốc: `raw_train/val/test`

Tái sử dụng

```

1  # Tải các bộ dữ liệu gốc đã được xử lý
2  X_train, y_train = read_csv("splits/raw_train.csv")
3  X_val, y_val = read_csv("splits/raw_val.csv")
4  X_test, y_test = read_csv("splits/raw_test.csv")

```

và các tập fe: `fe_train/val/test` để chuẩn bị cho những thực nghiệm bên dưới.

Tái sử dụng

```

1  # Tải các bộ dữ liệu được áp dụng kỹ thuật tạo đặc trưng
2  X_train_fe, y_train_fe = read_csv("splits/fe_train.csv")
3  X_val_fe, y_val_fe = read_csv("splits/fe_val.csv")
4  X_test_fe, y_test_fe = read_csv("splits/fe_test.csv")

```

Theo sau là 2 hàm `evaluate_val` và `evaluate_test`, mục đích chung của 2 này trong toàn bộ các mô hình trong dự án này là:

- Hàm `evaluate_val`: huấn luyện trên tập train và đánh giá trên tập val,
- Hàm `evaluate_test`: đánh giá mô hình đã huấn luyện trên tập test.

```

1  def evaluate_val(X_train, y_train, X_val, y_val):
2      # Huấn luyện Naive Bayes
3      nbc_model = GaussianNB()
4      nbc_model.fit(X_train, y_train)
5
6      # Dự đoán và đánh giá trên tập val
7      nb_pred = nbc_model.predict(X_val)
8      nb_accuracy = accuracy_score(y_val, nb_pred)
9
10     print(f"Độ chính xác Naive Bayes: {nb_accuracy:.4f}")
11     print("\nClassification Report:")
12     print(classification_report(y_val, nb_pred))
13     return nbc_model, nb_accuracy
14

```



```

15 def evaluate_test(nbc_model, X_test, y_test):
16     # Dự đoán và đánh giá trên tập test
17     nb_test_pred = nbc_model.predict(X_test)
18     nb_test_accuracy = accuracy_score(y_test, nb_test_pred)
19
20     print(f"Độ chính xác Naive Bayes trên tập test: {nb_test_accuracy:.4f}")
21     print("\nClassification Report:")
22     print(classification_report(y_test, nb_test_pred))
23     return nb_test_accuracy

```

Khi này, tiến hành huấn luyện và đánh giá Naive Bayes trên các tập gốc, lưu lại độ chính xác trên val và test.

```

1 # Huấn luyện và đánh giá với dữ liệu gốc
2 model, accuracy = evaluate_val(X_train, y_train, X_val, y_val)
3 test_accuracy = evaluate_test(model, X_test, y_test)

```

Tương tự như bước trên cho tập fe.

```

1 # Huấn luyện và đánh giá với kỹ thuật tạo đặc trưng
2 model_fe, accuracy_fe = evaluate_val(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
3 test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)

```

Cuối cùng, vẽ biểu đồ cột so sánh độ chính xác (acc) trên tập val và test cho hai kịch bản (tập gốc và tập fe).

Tái sử dụng

```

1 plt.rcParams["font.family"] = "DejaVu Serif"
2 labels = ["Original Dataset", "Feature Engineering Dataset"]
3 val_accs = [accuracy, accuracy_fe]
4 test_accs = [test_accuracy, test_accuracy_fe]
5
6 x = np.arange(len(labels))
7 width = 0.3
8
9 fig, ax = plt.subplots(figsize=(5, 5))
10 rects1 = ax.bar(x - width/2, val_accs, width,
11                label="Validation Accuracy",
12                color="tab:blue", edgecolor="black", linewidth=1.2)
13 rects2 = ax.bar(x + width/2, test_accs, width,
14                label="Test Accuracy",
15                color="tab:red", edgecolor="black", linewidth=1.2)
16
17 ax.set_ylim(0.5, 1.05)
18 ax.set_ylabel("Accuracy")
19 ax.set_title("Acc trên tập Val và Test")
20 ax.set_xticks(x)

```

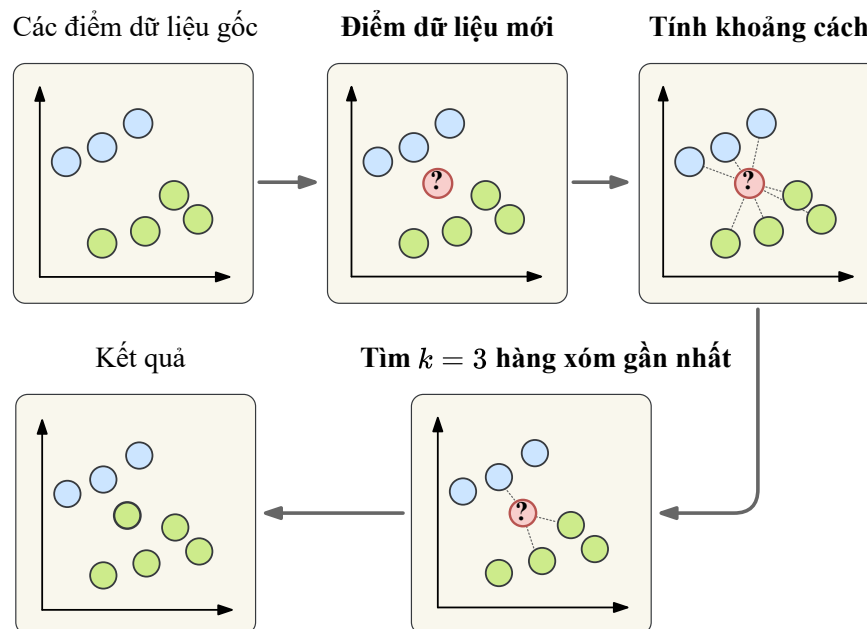
```

21 ax.set_xticklabels(labels)
22 ax.legend(ncol=2, loc="upper center")
23
24 def autolabel(rects):
25     for rect in rects:
26         h = rect.get_height()
27         ax.annotate(f"{h:.2f}", xy=(rect.get_x()+rect.get_width()/2, h),
28                 ha="center", va="bottom")
29
30 autolabel(rects1)
31 autolabel(rects2)
32
33 fig.tight_layout()
34 plt.show()

```

II.4. K-Nearest Neighbors

II.4.1. Giới thiệu



Hình 9: Tổng quan các bước trong mô hình KNN.

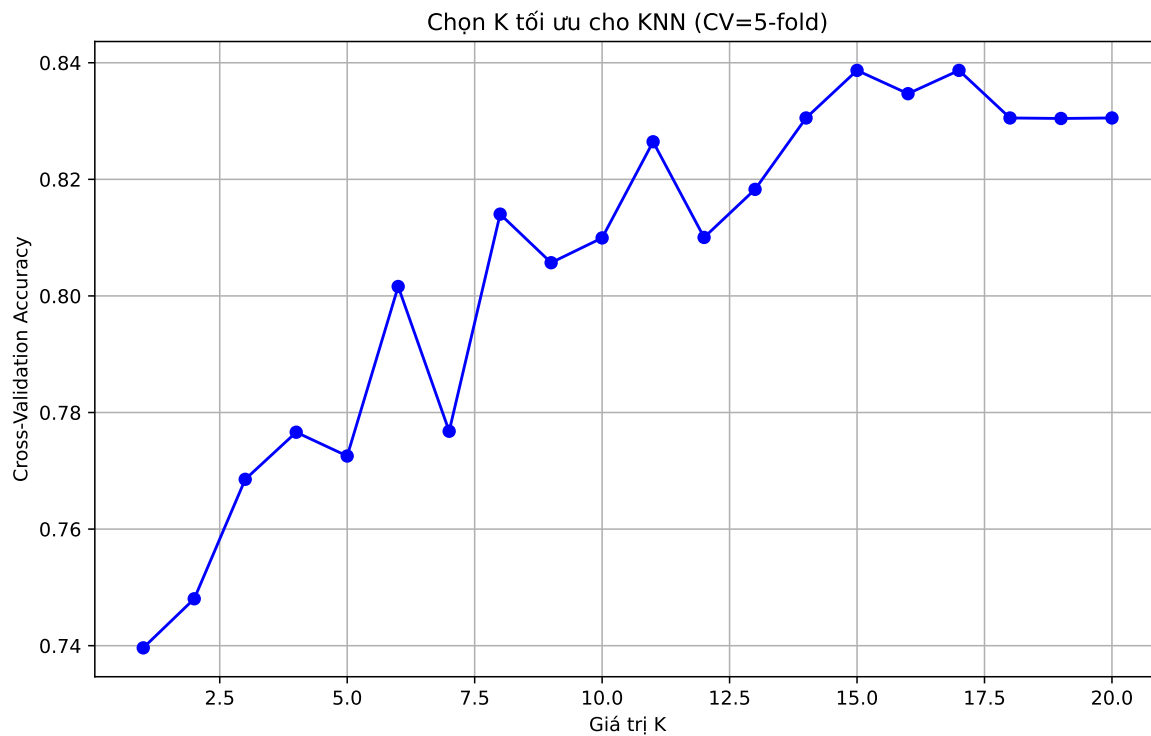
Tiếp theo là K-Nearest Neighbors (KNN), một mô hình học có giám sát và phi tham số (non-parametric). Thuật toán này không học một hàm mục tiêu cụ thể, mà lưu trữ toàn bộ tập huấn luyện để sử dụng khi dự đoán. Nguyên tắc hoạt động dựa trên giả định rằng các điểm dữ liệu có đặc trưng tương tự nhau sẽ có xu hướng nằm gần nhau trong không gian đặc trưng. Khi phân loại một mẫu mới, KNN xác định K điểm gần nhất trong tập huấn luyện (các “hàng xóm”) dựa trên một thước đo khoảng cách, sau đó gán nhãn dựa trên đa số phiếu của các hàng xóm này.

Lựa chọn giá trị K là yếu tố quan trọng: K quá nhỏ khiến mô hình nhạy cảm với nhiễu, trong khi K quá lớn có thể làm mờ ranh giới giữa các lớp.

II.4.2. Triển khai

Với mô hình KNN, tái sử dụng các khối code cũ và điều chỉnh một số hàm chính như sau:

- Bổ sung hàm tìm k tối ưu theo phương pháp Stratified K-Fold Cross Validation,
- Điều chỉnh hàm `evaluate_val` và `evaluate_test` cho KNN.



Hình 10: Biểu đồ xác định K tối ưu cho KNN (CV=5-fold).

```

1 def find_optimal_k(X_train, y_train, X_val=None, y_val=None,
2                   k_range=range(1, 21), cv_splits=5):
3     # Tạo bộ chia dữ liệu cross-validation
4     cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
5     k_scores = []
6
7     # Đánh giá bằng cross-validation trên tập train
8     for k in k_range:
9         knn = KNeighborsClassifier(n_neighbors=k)
10        cv_score = cross_val_score(
11            knn, X_train, y_train,
12            cv=cv, scoring="accuracy", n_jobs=-1)
13        k_scores.append(cv_score.mean())
14
15     # Vẽ biểu đồ chọn k theo CV

```

```

16 plt.figure(figsize=(10, 6))
17 plt.plot(list(k_range), k_scores, "bo-")
18 plt.title(f"Chọn K tối ưu cho KNN (CV={cv_splits}-fold)")
19 plt.xlabel("Giá trị K")
20 plt.ylabel("Cross-Validation Accuracy")
21 plt.grid(True)
22 plt.show()
23
24 # K tối ưu theo CV
25 optimal_k = list(k_range)[int(np.argmax(k_scores))]
26 print(f"K tối ưu (CV): {optimal_k}")
27 return optimal_k
28
29 def evaluate_val(X_train, y_train, X_val, y_val, optimal_k):
30     # Huấn luyện KNN với k tối ưu
31     print(f"Huấn luyện KNN với k tối ưu: {optimal_k}")
32     knn_model = KNeighborsClassifier(n_neighbors=optimal_k)
33     knn_model.fit(X_train, y_train)
34
35     # Dự đoán và đánh giá trên tập val
36     knn_pred = knn_model.predict(X_val)
37     knn_accuracy = accuracy_score(y_val, knn_pred)
38
39     print(f"\nĐộ chính xác KNN trên tập validation: {knn_accuracy:.2f}")
40     print("Classification Report:")
41     print(classification_report(y_val, knn_pred))
42     return knn_model, knn_accuracy
43
44 def evaluate_test(knn_model, X_test, y_test):
45     # Dự đoán và đánh giá trên tập test
46     knn_test_pred = knn_model.predict(X_test)
47     knn_test_accuracy = accuracy_score(y_test, knn_test_pred)
48
49     print(f"\nĐộ chính xác KNN trên tập test: {knn_test_accuracy:.2f}")
50     print("Classification Report:")
51     print(classification_report(y_test, knn_test_pred))
52     return knn_test_accuracy

```

Trong phần huấn luyện mô hình, thực hiện tương tự như mô hình trước nhưng bổ sung thêm việc tìm k tối ưu và sau đó huấn luyện mô hình KNN với giá trị k này.

```

1 k_optimal = find_optimal_k(X_train, y_train, X_val, y_val)
2 model, accuracy = evaluate_val(X_train, y_train, X_val, y_val, k_optimal)
3 test_accuracy = evaluate_test(model, X_test, y_test)
4
5 ###
6
7 k_optimal = find_optimal_k(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
8 model_fe, accuracy_fe = evaluate_val(
9     X_train_fe, y_train_fe,
10    X_val_fe, y_val_fe,

```

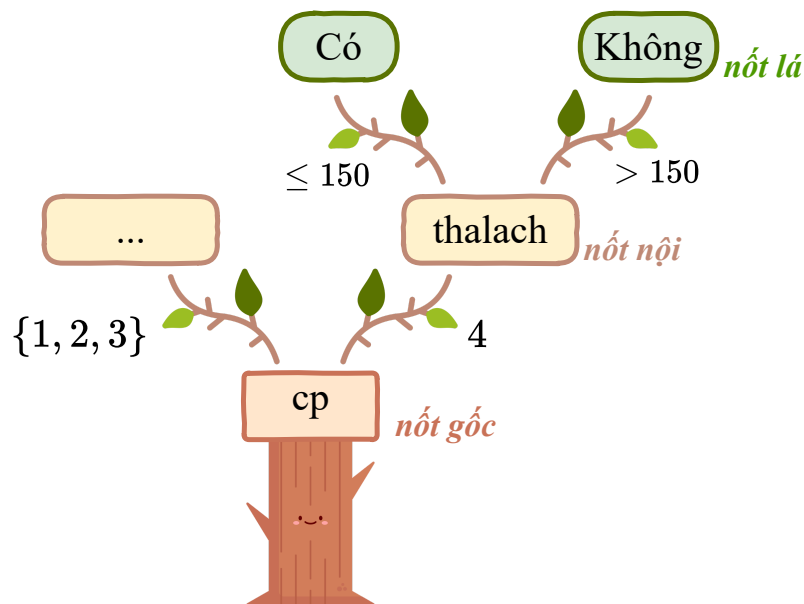
```

11     k_optimal
12 )
13 test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)

```

II.5. Decision Tree

II.5.1. Giới thiệu



Hình 11: Minh họa mô hình cây quyết định bằng các thuộc tính của bài toán dự đoán bệnh tim.

Sau KNN, chúng ta chuyển sang Decision Tree, một mô hình học có giám sát sử dụng cấu trúc dạng cây để đưa ra quyết định phân loại. Mô hình chia nhỏ không gian đặc trưng thành các vùng bằng cách áp dụng các quy tắc “nếu-thì” tại mỗi nút. Mỗi **nút nội** biểu diễn một phép kiểm tra trên một đặc trưng, mỗi nhánh thể hiện kết quả của phép kiểm tra đó, và mỗi **nút lá** đại diện cho nhãn dự đoán. Việc xây dựng cây dựa trên việc chọn đặc trưng và ngưỡng phân chia tối ưu tại mỗi bước, nhằm tối đa hóa độ thuần khiết của các nút con, thường đo bằng các chỉ số như **Gini Impurity** hoặc **Information Gain**. Nhờ tính trực quan và khả năng diễn giải cao, Decision Tree là một trong những mô hình nền tảng và dễ hiểu nhất trong học máy.

II.5.2. Triển khai

Với mô hình DT, tái sử dụng các khối code cũ và điều chỉnh một số hàm chính như sau:

- Bổ sung hàm tìm độ sâu (Depth) tối ưu theo phương pháp Stratified K-Fold Cross Validation.
- Điều chỉnh hàm `evaluate_val` và `evaluate_test` cho DT.
- Bổ sung hàm hiển thị các đặc trưng có ảnh hưởng với mô hình.

```

1 def find_optimal_depth(X_train, y_train,
2                       depth_range=range(1, 11), cv_splits=5, min_depth=2):
3     # Tạo bộ chia dữ liệu cross-validation
4     cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
5     depth_scores = []
6
7     # Đánh giá bằng cross-validation trên tập train
8     for depth in range(min_depth, depth_range.stop):
9         dt = DecisionTreeClassifier(max_depth=depth, random_state=SEED)
10        cv_score = cross_val_score(
11            dt, X_train, y_train,
12            cv=cv, scoring="accuracy", n_jobs=-1)
13        depth_scores.append(cv_score.mean())
14
15    # Vẽ biểu đồ chọn k theo CV
16    plt.figure(figsize=(10, 6))
17    plt.plot(list(range(min_depth, depth_range.stop)), depth_scores, "bo-")
18    plt.title(f"Chọn Depth tối ưu cho DT (CV={cv_splits}-fold)")
19    plt.xlabel("Giá trị Depth")
20    plt.ylabel("Cross-Validation Accuracy")
21    plt.grid(True)
22    plt.show()
23
24    # K tối ưu theo CV
25    optimal_depth = list(range(min_depth, depth_range.stop))[int(np.argmax(depth_scores
26                                                                    ))]
27    print(f"Depth tối ưu (CV): {optimal_depth}")
28    return optimal_depth
29
30 def evaluate_val(X_train, y_train, X_val, y_val, optimal_depth):
31     # Huấn luyện với độ sâu tối ưu
32     dt_model = DecisionTreeClassifier(max_depth=optimal_depth, random_state=SEED)
33     dt_model.fit(X_train, y_train)
34
35     # Dự đoán và đánh giá trên tập val
36     dt_pred = dt_model.predict(X_val)
37     dt_accuracy = accuracy_score(y_val, dt_pred)
38     print(f"\nĐộ chính xác Decision Tree trên tập validation: {dt_accuracy:.2f}")
39
40     print("Classification Report:")
41     print(classification_report(y_val, dt_pred))
42     return dt_model, dt_accuracy
43
44 def evaluate_test(dt_model, X_test, y_test):
45     # Dự đoán và đánh giá trên tập test
46     dt_test_pred = dt_model.predict(X_test)
47     dt_test_accuracy = accuracy_score(y_test, dt_test_pred)
48     print(f"\nĐộ chính xác Decision Tree trên tập test: {dt_test_accuracy:.2f}")
49
50     print("Classification Report:")
51     print(classification_report(y_test, dt_test_pred))
52     return dt_test_accuracy
53
54 def display_feature_importance(dt_model, X_train):

```

```

54 # Hiển thị các đặc trưng có ảnh hưởng lớn
55 feature_importance = pd.DataFrame({
56     "feature": X_train.columns,
57     "importance": dt_model.feature_importances_
58 }).sort_values("importance", ascending=False)
59 print("\nXếp hạng Features quan trọng:")
60 display(feature_importance.head(10))

```

Phần huấn luyện này tương tự KNN thực hiện tương tự như mô hình trước nhưng bổ sung thêm việc tìm k tối ưu và sau đó huấn luyện mô hình KNN với giá trị k này.

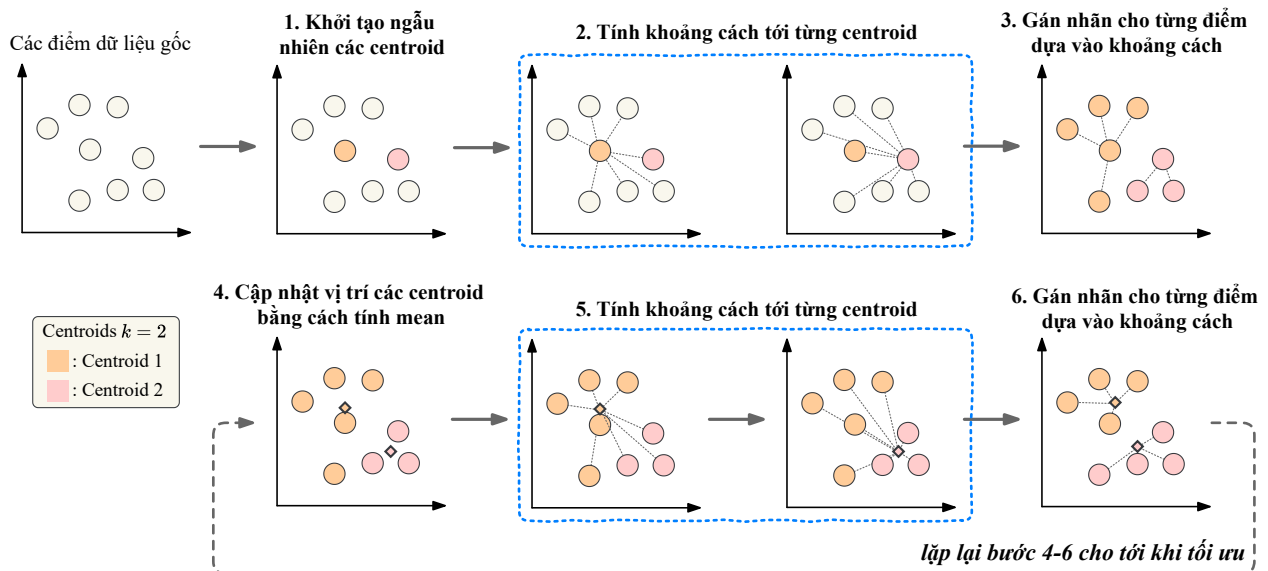
```

1 depth_optimal = find_optimal_depth(X_train, y_train, X_val, y_val)
2 model, accuracy = evaluate_val(X_train, y_train, X_val, y_val, depth_optimal)
3 test_accuracy = evaluate_test(model, X_test, y_test)
4 display_feature_importance(model, X_train)
5
6 ###
7
8 depth_optimal = find_optimal_depth(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
9 model_fe, accuracy_fe = evaluate_val(X_train_fe, y_train_fe, X_val_fe, y_val_fe,
                                     depth_optimal)
10 test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)
11 display_feature_importance(model_fe, X_train_fe)

```

II.6. K-means

II.6.1. Giới thiệu



Hình 12: Minh họa từng bước hoạt động của thuật toán k-means với $k = 2$.

Khác với các mô hình học có giám sát bên trên, K-means là một thuật toán học không giám sát được sử dụng phổ biến cho bài toán phân cụm dữ liệu. Ý tưởng chính của K-means là nhóm các điểm dữ liệu thành K cụm sao cho các điểm trong cùng một cụm có đặc trưng tương tự nhau và khác biệt so với các cụm khác.

Thuật toán bắt đầu bằng việc khởi tạo ngẫu nhiên K điểm làm centroid (tâm cụm), sau đó lặp lại hai bước chính:

1. Gán mỗi điểm dữ liệu vào cụm có centroid gần nhất và
2. cập nhật lại vị trí của các centroid dựa trên trung bình các điểm trong cụm.

Quá trình này tiếp tục cho đến khi vị trí các centroid hội tụ hoặc đạt điều kiện dừng.

II.6.2. Triển khai

```

1 def evaluate_val(X_train, y_train, X_val, y_val):
2     # Huấn luyện
3     kmean_model = KMeans(n_clusters=2, random_state=SEED, init="random")
4     kmean_model.fit(X_train)
5
6     # Dự đoán và đánh giá trên tập val
7     val_clusters = kmean_model.predict(X_val)
8     train_clusters = kmean_model.labels_
9     cluster_class_mapping = {
10         cluster_id: Counter(y_train[train_clusters == cluster_id]).most_common(1)[0][0]
11         for cluster_id in np.unique(train_clusters)
12     }
13     kmeans_pred = np.array([cluster_class_mapping[cluster_id]
14                             for cluster_id in val_clusters])
15     kmeans_accuracy = accuracy_score(y_val, kmeans_pred)
16
17     print(f"Độ chính xác KMeans (Binary Prediction): {kmeans_accuracy:.4f}")
18     print("\nClassification Report (KMeans Binary Prediction):")
19     print(classification_report(y_val, kmeans_pred))
20     return kmean_model, kmeans_accuracy
21
22 def evaluate_test(kmean_model, X_test, y_test):
23     # Dự đoán trên tập test
24     test_clusters = kmean_model.predict(X_test)
25     train_clusters = kmean_model.labels_
26     cluster_class_mapping = {
27         cluster_id: Counter(y_train[train_clusters == cluster_id]).most_common(1)[0][0]
28         for cluster_id in np.unique(train_clusters)
29     }
30     kmeans_test_pred = np.array([cluster_class_mapping[cluster_id]
31                                 for cluster_id in test_clusters])
32     kmeans_test_accuracy = accuracy_score(y_test, kmeans_test_pred)
33     print(f"Độ chính xác KMeans trên tập test: {kmeans_test_accuracy:.4f}")
34     print("\nClassification Report:")
35     print(classification_report(y_test, kmeans_test_pred))
36     return kmeans_test_accuracy

```


Phần huấn luyện K-Means thực hiện tương tự như các phần trước.

```

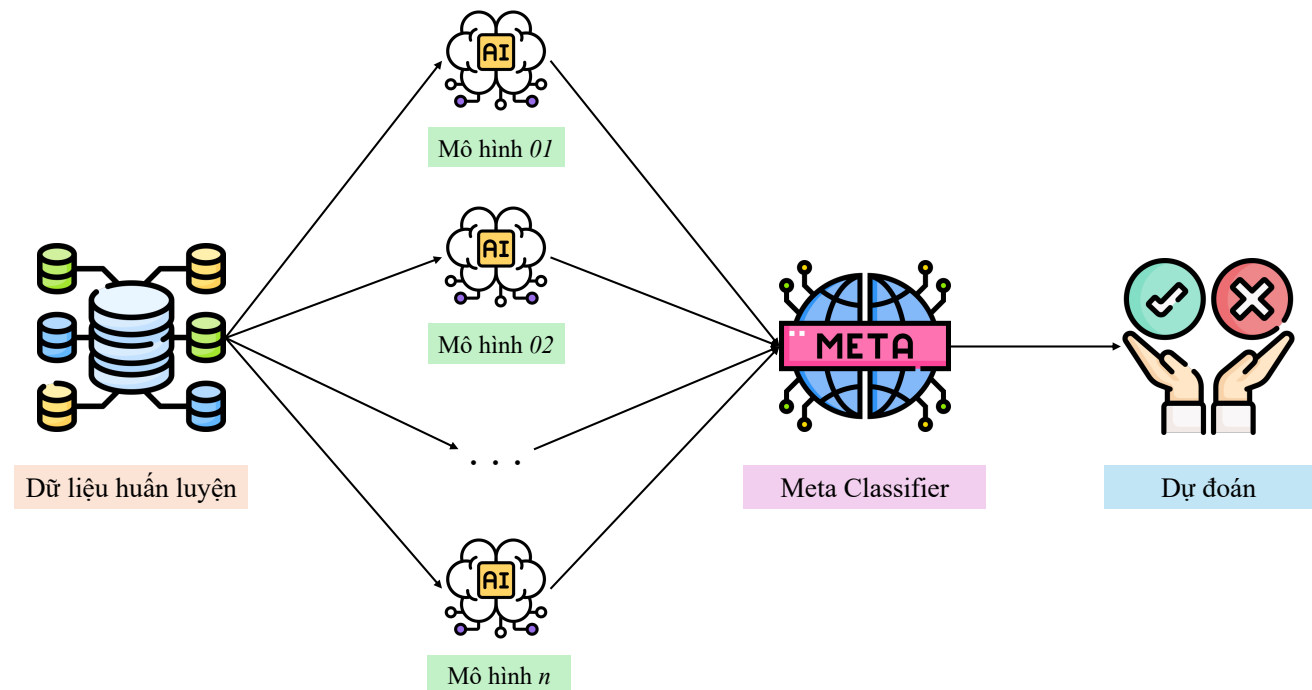
1 model, accuracy = evaluate_val(X_train, y_train, X_val, y_val)
2 test_accuracy = evaluate_test(model, X_test, y_test)
3
4 model_fe, accuracy_fe = evaluate_val(X_train_fe, y_train_fe, X_val_fe, y_val_fe)
5 test_accuracy_fe = evaluate_test(model_fe, X_test_fe, y_test_fe)

```

II.7. Cải tiến: Ensemble

II.7.1. Giới thiệu

Ensemble learning là một kỹ thuật trong học máy nhằm kết hợp nhiều mô hình dự đoán để tạo thành một mô hình mạnh hơn. Ý tưởng chính là tận dụng ưu điểm của từng mô hình đơn lẻ, từ đó giảm sai số và tăng độ ổn định trong quá trình dự đoán. Chính bởi lẽ đó, chúng ta có thể tận dụng các mô hình đã huấn luyện từ trước trong project để ứng dụng kỹ thuật này nhằm làm tăng độ chính xác tổng thể của toàn hệ thống.



Hình 13: Minh họa ý tưởng cơ bản của kỹ thuật ensemble learning tận dụng n mô hình AI đã được huấn luyện.

Ensemble learning bao gồm rất nhiều những kỹ thuật con xoay quanh, song trong phạm vi của project này, chúng ta sử dụng phương pháp Stacking, kết hợp ba mô hình cơ bản (KNN, Decision Tree và Naive Bayes) và dùng một bộ phân loại cuối cùng để đưa ra kết quả. Cách tiếp cận này được kỳ vọng sẽ cải thiện hiệu năng so với từng mô hình riêng lẻ.

II.7.2. Triển khai

Đầu tiên, chúng ta khởi tạo mô hình Stacking Ensemble, trong đó ba mô hình con (KNN, Decision Tree và Naive Bayes) đóng vai trò là các bộ phân loại cơ sở. Mô hình KNN được chọn làm bộ phân loại cuối cùng để tổng hợp kết quả.

```

1 def init_model():
2     # Khởi tạo mô hình Stacking Ensemble
3     ensemble_model = StackingClassifier(
4         estimators=[
5             ("knn", KNeighborsClassifier()),
6             ("dt", DecisionTreeClassifier(random_state=SEED)),
7             ("nb", GaussianNB()),
8         ],
9         final_estimator=KNeighborsClassifier(),
10        stack_method="predict_proba",
11        passthrough=False
12    )
13    display(ensemble_model)
14    return ensemble_model

```

Tiếp theo, chúng ta định nghĩa các hàm đánh giá, bao gồm huấn luyện và kiểm tra trên tập validation, cũng như đánh giá hiệu năng cuối cùng trên tập test. Các hàm này trả về độ chính xác cùng báo cáo chi tiết về precision, recall và F1-score.

```

1 def evaluate_ensemble(ensemble_model, X_train, y_train, X_val, y_val):
2     ensemble_model.fit(X_train, y_train)
3     ensemble_pred = ensemble_model.predict(X_val)
4     ensemble_accuracy = accuracy_score(y_val, ensemble_pred)
5     print(f"Độ chính xác Ensemble Model: {ensemble_accuracy:.4f}")
6     print("\nClassification Report:")
7     print(classification_report(y_val, ensemble_pred))
8     return ensemble_accuracy
9
10 def evaluate_ensemble_test(ensemble_model, X_test, y_test):
11     ensemble_test_pred = ensemble_model.predict(X_test)
12     ensemble_test_accuracy = accuracy_score(y_test, ensemble_test_pred)
13     print(f"Độ chính xác Ensemble trên tập test: {ensemble_test_accuracy:.4f}")
14     print("\nClassification Report:")
15     print(classification_report(y_test, ensemble_test_pred))
16     return ensemble_test_accuracy

```

Sau khi đã xây dựng các hàm, chúng ta tiến hành huấn luyện mô hình Ensemble trên bộ dữ liệu gốc, đồng thời đánh giá hiệu năng trên cả tập validation và tập test.

```

1 ensemble_model = init_model()
2 accuracy = evaluate_ensemble(
3     ensemble_model,
4     X_train,

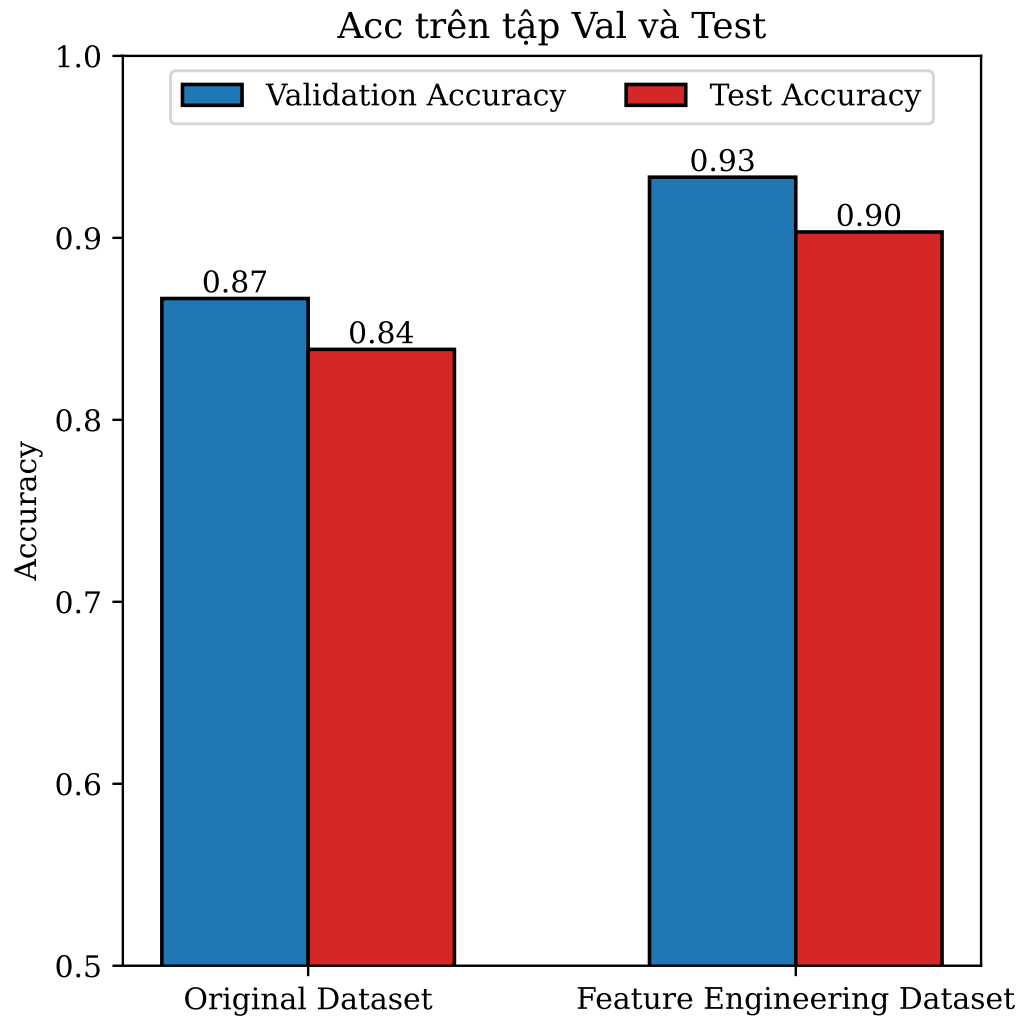
```

```
5     y_train,  
6     X_val,  
7     y_val  
8 )  
9 test_accuracy = evaluate_ensemble_test(  
10     ensemble_model,  
11     X_test,  
12     y_test  
13 )
```

Tương tự, chúng ta cũng lặp lại quy trình với bộ dữ liệu đã được xử lý đặc trưng (feature engineering). Điều này giúp đánh giá mức độ cải thiện hiệu năng của kỹ thuật ensemble khi dữ liệu đầu vào được tinh chỉnh tốt hơn.

```
1 ensemble_model_fe = init_model()  
2 accuracy_fe = evaluate_ensemble(  
3     ensemble_model_fe,  
4     X_train_fe,  
5     y_train_fe,  
6     X_val_fe,  
7     y_val_fe  
8 )  
9 test_accuracy_fe = evaluate_ensemble_test(  
10     ensemble_model_fe,  
11     X_test_fe,  
12     y_test_fe  
13 )
```

Biểu đồ so sánh ở Hình 14 minh họa sự khác biệt về độ chính xác giữa hai cách tiếp cận: sử dụng dữ liệu gốc và dữ liệu đã được xử lý đặc trưng. Có thể nhận thấy rằng việc áp dụng kỹ thuật feature engineering kết hợp với Ensemble giúp cải thiện hiệu năng một cách rõ rệt.



Hình 14: Biểu đồ so sánh mô hình Ensemble (Stacking) trên bộ dữ liệu gốc và bộ được xử lý đặc trưng.

III. Câu hỏi trắc nghiệm

1. Trong phần Naive Bayes Classifier, mô hình giả định rằng:
 - (a) Tất cả đặc trưng y tế đều phụ thuộc tuyến tính vào tuổi.
 - (b) Mọi đặc trưng phải được chuẩn hóa trước khi tính toán.
 - (c) Cholesterol và huyết áp độc lập có điều kiện khi biết bệnh nhân mắc hay không mắc bệnh tim.
 - (d) Mỗi đặc trưng chỉ có thể nhận 2 giá trị rời rạc.
2. Trong phần cài đặt code KNN, giá trị K được chọn bằng cách:
 - (a) Dùng GridSearchCV trên toàn bộ dữ liệu.
 - (b) Chọn K bằng cách thử nghiệm ngẫu nhiên nhiều lần.
 - (c) Luôn cố định $K = 5$ theo tài liệu tham khảo.
 - (d) Tối ưu bằng Stratified K-Fold Cross Validation với k từ 1 đến 20.
3. Khi huấn luyện Decision Tree trong bài, tác giả dùng chỉ số nào để chọn thuộc tính chia tách?
 - (a) Entropy hoặc Gini Impurity để tối đa hóa Information Gain.
 - (b) Mean Squared Error vì đây là bài toán hồi quy.
 - (c) Cosine Similarity để so sánh đặc trưng.
 - (d) Precision-Recall Curve trên tập validation.
4. Với việc thuật toán K-means được cài đặt số cụm bằng 2, điều này phản ánh ý nghĩa gì?
 - (a) Hai trạng thái sức khỏe tim mạch: có bệnh và không bệnh.
 - (b) Hai nhóm bệnh nhân nam và nữ.
 - (c) Hai loại chỉ số lâm sàng: cholesterol và huyết áp.
 - (d) Hai bộ dữ liệu raw và feature engineering.
5. Trong phần Ensemble, bài viết sử dụng `StackingClassifier` với ba mô hình con là:
 - (a) Logistic Regression, SVM, Random Forest.
 - (b) KNN, Naive Bayes, K-means.
 - (c) Decision Tree, K-means, SVM.
 - (d) KNN, Decision Tree, Naive Bayes.
6. Trong phần Decision Tree, hàm `display_feature_importance()` có vai trò gì?
 - (a) Hiển thị phân phối nhãn trong tập huấn luyện.
 - (b) Xếp hạng các đặc trưng quan trọng nhất dựa trên độ đóng góp vào việc phân loại.
 - (c) Tự động chọn độ sâu tối ưu cho cây.

(d) Chuẩn hóa các đặc trưng đầu vào trước khi huấn luyện.

7. Trong tập dữ liệu Cleveland, thuộc tính *ca* có ý nghĩa gì?

- (a) Loại đau ngực (1–4).
- (b) Thalassemia (3,6,7).
- (c) Điện tâm đồ lúc nghỉ (0–2).
- (d) Số mạch máu chính được nhuộm màu (0–3).

8. Đoạn code sau trong phần KNN có mục đích gì?

```
1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

- (a) Khởi tạo mô hình KNN với $k=5$.
- (b) Chia dữ liệu thành 5 folds cân bằng theo nhãn để cross-validation, giữ kết quả nhất quán nhờ seed.
- (c) Ngăn chặn overfitting bằng cách bỏ bớt dữ liệu.
- (d) Chuẩn hóa các đặc trưng về cùng khoảng giá trị.

9. Trong phần Naive Bayes, lệnh `classification_report(y_val, nb_pred)` trả về:

- (a) Precision, Recall, F1-score cho từng lớp (mắc bệnh tim / không mắc bệnh tim).
- (b) Chỉ duy nhất Accuracy trên toàn bộ tập validation.
- (c) Biểu đồ histogram của dự đoán.
- (d) Giá trị K tối ưu được chọn.

10. Trong triển khai K-means, đoạn code:

```
1 cluster_class_mapping = {
2     cluster_id: Counter(y_train[train_clusters == cluster_id]).most_common(1)[0][0]
3     for cluster_id in np.unique(train_clusters)
4 }
```

có ý nghĩa là:

- (a) Ánh xạ mỗi cụm thành nhãn “có bệnh tim” hoặc “không bệnh” dựa trên nhãn chiếm đa số trong cụm train.
- (b) Chuẩn hóa dữ liệu trước khi phân cụm.
- (c) Tự động chọn số cụm tối ưu.
- (d) Loại bỏ các điểm ngoại lai trong tập train.

IV. Tài liệu tham khảo

- [1] PCR, *World Heart Day 2023: Reducing the burden of cardiovascular disease globally: Beyond stents and balloons!*, en. [Online]. Available: <https://www.pcronline.com/News/Whats-new-on-PCRonline/2023/World-Heart-Day-2023-Reducing-burden-cardiovascular-disease-globally-beyond-stents-balloons>.
- [2] *Cardiovascular Disease / NCD Alliance*, en, Oct. 2023. [Online]. Available: <https://ncdaliance.org/explore-ncds/ncds/cardiovascular-disease>.
- [3] *World Heart Report 2023: Full Report*, en-GB. [Online]. Available: <https://world-heart-federation.org/resource/world-heart-report-2023/>.
- [4] A. O. Mocumbi, “Cardiovascular Health Care in Low- and Middle-Income Countries”, *Circulation*, vol. 149, no. 8, pp. 557–559, Feb. 2024, Publisher: American Heart Association. DOI: 10.1161/CIRCULATIONAHA.123.065717. [Online]. Available: <https://www.ahajournals.org/doi/10.1161/CIRCULATIONAHA.123.065717>.
- [5] P. Shinde et al., “A Survey on Machine Learning Techniques for Heart Disease Prediction”, en, *SN Computer Science*, vol. 6, no. 4, p. 334, Apr. 2025, ISSN: 2661-8907. DOI: 10.1007/s42979-025-03860-2. [Online]. Available: <https://doi.org/10.1007/s42979-025-03860-2>.
- [6] S. W. P. M. Janosi Andras et al., *Heart Disease*, UCI Machine Learning Repository, DOI: <https://doi.org/10.24432/C52P4X>, 1989.
- [7] R. C. Detrano et al., “International application of a new probability algorithm for the diagnosis of coronary artery disease.”, *The American journal of cardiology*, vol. 64 5, pp. 304–10, 1989. [Online]. Available: <https://api.semanticscholar.org/CorpusID:23545303>.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Hint:** Các file code gợi ý có thể được tải tại [đây](#).
3. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại [đây](#).
4. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại [đây](#).
5. **Rubric:**

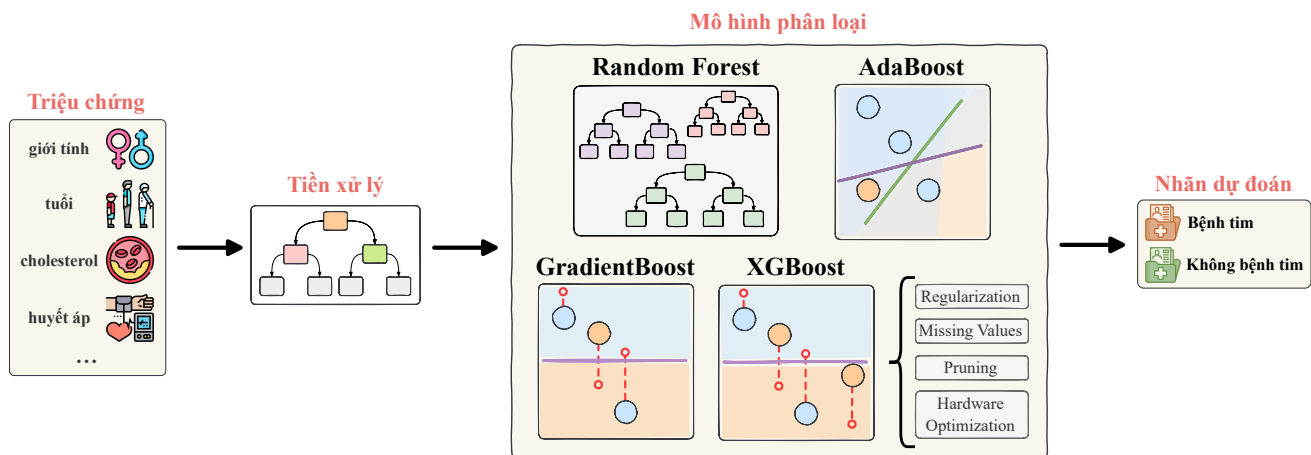
Mục	Kiến Thức	Đánh Giá
I.	- Hiểu bối cảnh bệnh tim mạch và tầm quan trọng của chẩn đoán sớm. - Nắm được thông tin về bộ dữ liệu Cleveland Heart Disease. - Biết các đặc trưng y tế chính (age, sex, cp, chol, trestbps, ...).	- Trình bày được lý do chọn bộ dữ liệu Cleveland. - Nhận biết và giải thích ý nghĩa các đặc trưng trong bộ dữ liệu. - Hiểu được mối liên hệ giữa dữ liệu y tế và mô hình học máy.
II.	- Kiến thức về xử lý đặc trưng (feature engineering). - Kiến thức về các mô hình cơ bản: Naive Bayes, KNN, Decision Tree. - Kiến thức về mô hình không giám sát K-means. - Kiến thức về Ensemble Learning (Stacking).	- Hiểu giả định của Naive Bayes và khi nào phù hợp. - Nhận biết vai trò của K trong KNN và độ sâu trong Decision Tree. - Giải thích nguyên lý K-means và lý do chọn $k = 2$. - Trình bày lợi ích của Ensemble trong việc kết hợp nhiều mô hình.
III.	- Kiến thức về triển khai code với scikit-learn. - Kỹ năng sử dụng cross-validation để tìm tham số tối ưu. - Biết cách đánh giá mô hình bằng accuracy, classification report, biểu đồ.	- Viết và hiểu được các hàm <code>evaluate_val</code>, <code>evaluate_test</code>. - Hiểu mục đích của StratifiedKFold trong tìm K tối ưu. - Giải thích được ý nghĩa kết quả accuracy và các chỉ số Precision, Recall, F1. - Biết so sánh hiệu quả giữa tập dữ liệu gốc và tập đã xử lý đặc trưng.

Project 3.1: Heart Disease Diagnosis Using Machine Learning

Dương Đình Thắng Vũ Yến Linh Nguyễn Anh Khôi
Dương Trường Bình Đinh Quang Vinh

I. Giới thiệu

Ở phần trước, chúng ta đã đặt những “viên gạch nền” cho bài toán dự đoán bệnh tim: hiểu cấu trúc dữ liệu Cleveland, chuẩn hóa quy trình tiền xử lý, áp dụng kỹ thuật tạo đặc trưng và xây dựng các mô hình Machine Learning (ML) cơ bản như Naive Bayes, K-Nearest Neighbors, Decision Tree, thậm chí thử nghiệm một biến thể ensemble cơ bản. Kết quả cho thấy các mô hình truyền thống, khi được huấn luyện bằng dữ liệu đã xử lý cẩn thận, có thể đạt hiệu năng đáng khích lệ và giúp ta đọc vị bài toán khá hiệu quả.



Hình 1: Pipeline Project Cleveland Heart Disease Diagnosis.

Bước sang phần 2 này, chúng ta tiếp tục nâng cấp cho hệ thống bằng những kỹ thuật ensemble nâng cao với những phương pháp vốn rất mạnh trên dữ liệu bảng:

- Random Forest đại diện cho họ Bagging, kết hợp nhiều cây độc lập để giảm phương sai và tăng ổn định.

- AdaBoost và Gradient Boosting đại diện cho họ Boosting, nơi các mô hình liên tiếp “sửa sai” lẫn nhau.
- XGBoost hiện thực Boosting một cách tối ưu hơn với nhiều cải tiến về regularization và hiệu năng tính toán.

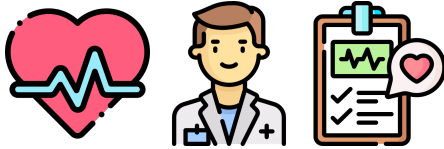
Mục lục

I.	Giới thiệu	1
II.	Cài đặt chương trình	4
II.1.	Bộ dữ liệu	4
II.2.	Cải thiện đặc trưng	5
II.3.	Random Forest	14
II.4.	AdaBoost	21
II.5.	Gradient Boosting	25
II.6.	XGBoost	29
II.7.	Tổng hợp kết quả	33
II.8.	Cài đặt UI và Deploy	33
III.	Câu hỏi trắc nghiệm	40
IV.	Tài liệu tham khảo	44
	Phụ lục	45

Trong toàn bộ dự án này, mọi điều kiện thực nghiệm được giữ nguyên, nhằm tái sử dụng các tập dữ liệu train/val/test và hai cấu hình dữ liệu gồm bản gốc và bản đã xử lý đặc trưng (feature engineering). Chúng ta cũng sử dụng bộ chỉ số quen thuộc như Accuracy, Precision, Recall, F1. Sự thay đổi chính sẽ tập trung vào việc xây dựng một bộ dữ liệu mới bằng mô hình Decision Tree; và huấn luyện các bộ dữ liệu này trên các mô hình Ensemble nâng cao. Chương tiếp theo sẽ trình bày cách triển khai thực nghiệm trong dự án này.

II. Cài đặt chương trình

II.1. Bộ dữ liệu



Cleveland Heart Disease Diagnosis

Hình 2: Bộ dữ liệu dự đoán bệnh tim Cleveland Heart Disease Diagnosis.

Để đảm bảo tính nhất quán và so sánh hiệu quả giữa các mô hình, dự án này tiếp tục sử dụng bộ dữ liệu **Cleveland Heart Disease** [1] đã được giới thiệu chi tiết ở dự án trước. Bộ dữ liệu bao gồm thông tin y tế của **303 bệnh nhân**, với 13 đặc trưng đầu vào (như tuổi, giới tính, cholesterol, v.v.) và một biến mục tiêu để chẩn đoán bệnh nhân có mắc bệnh tim hay không. Bảng dưới đây tóm tắt lại ý nghĩa của từng đặc trưng.

Bảng 1: Một số mẫu dữ liệu từ bộ Cleveland Heart Disease.

age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0
67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	1
67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0
41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0

Bảng mô tả đặc trưng bộ dữ liệu Cleveland Heart Disease

Đặc trưng	Mô tả và giá trị mã hóa
age	Tuổi của bệnh nhân (năm).
sex	Giới tính (1 = nam, 0 = nữ).
cp	Loại đau ngực. (1 = đau thắt ngực điển hình, 2 = không điển hình, 3 = đau không do tim, 4 = không triệu chứng)
trestbps	Huyết áp tâm thu lúc nghỉ (mmHg).
chol	Nồng độ cholesterol huyết thanh (mg/dL).
fbs	Đường huyết lúc đói > 120 mg/dL. (1 = đúng, 0 = sai)
restecg	Điện tâm đồ lúc nghỉ. (0 = bình thường, 1 = bất thường ST-T, 2 = phì đại thất trái)
thalach	Nhịp tim tối đa đạt được (lần/phút).
exang	Đau ngực khi gắng sức. (1 = có, 0 = không)
oldpeak	Mức độ trầm ST do gắng sức so với nghỉ.
slope	Độ dốc đoạn ST. (1 = dốc lên, 2 = bằng phẳng, 3 = dốc xuống)
ca	Số mạch máu chính được nhuộm màu (0–3).
thal	Thalassemia. (3 = bình thường, 6 = tổn thương cố định, 7 = tổn thương có thể đảo ngược)
num	Nhân mục tiêu. (0 = không bệnh, 1–4 = có bệnh)

Các giá trị rời rạc được mã hóa trong ngoặc để phục vụ huấn luyện mô hình.

II.2. Cải thiện đặc trưng

Trong phần này, chúng ta thực hiện kỹ thuật xử lý đặc trưng (Feature Engineering - FE) tương tự như dự án trước để nhận được 2 bộ dữ liệu gồm: bộ gốc và bộ FE (cả 2 tập đều có 13 cột/đặc trưng). Sau đó, sử dụng mô hình Decision Tree (DT) để tìm các đặc trưng quan trọng và tạo bộ dữ liệu mới với $K = 10$ đặc trưng quan trọng nhất. Cuối cùng, ta sẽ có tổng cộng 4 bộ dữ liệu: bộ gốc, bộ FE, bộ gốc áp dụng DT, bộ FE áp dụng DT.

Công việc trước tiên là tải các thư viện và thiết lập giá trị ngẫu nhiên, tạo nền tảng vững

chắc cho quá trình xử lý.

Tải các thư viện cần thiết

```

1 import os
2 import json
3 import random
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7
8 from pathlib import Path
9 from sklearn.pipeline import Pipeline
10 from sklearn.impute import SimpleImputer
11 from sklearn.compose import ColumnTransformer
12 from sklearn.tree import DecisionTreeClassifier
13 from sklearn.model_selection import train_test_split
14 from sklearn.base import BaseEstimator, TransformerMixin
15 from sklearn.feature_selection import mutual_info_classif
16 from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler
17
18 os.environ['PYTHONHASHSEED'] = '42'
19 np.random.seed(42)
20 random.seed(42)

```

Sau đó, tải tập dữ liệu Cleveland được lưu trữ trên Google Drive.

Tải bộ dữ liệu gốc (chưa được xử lý)

```

1 # https://drive.google.com/file/d/16HPyuXWXPtt5g3xvS_kR_wXAfjpR1Ju/view?usp=sharing
2 !gdown 16HPyuXWXPtt5g3xvS_kR_wXAfjpR1Ju

```

Tiếp đến, tập dữ liệu được đọc và cấu trúc hóa với các cột được đặt tên, dữ liệu số được chuyển đổi, các giá trị thiếu được xác định và cột mục tiêu dự đoán được mã hóa nhị phân để sẵn sàng cho phân tích.

Đọc và cấu trúc hóa tập dữ liệu

```

1 DATA_PATH = 'cleveland.csv'
2 COLUMNS = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
3             'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
4
5 numeric_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
6 categorical_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
7
8 K_features = 10
9 raw = pd.read_csv(DATA_PATH, header=None)
10 raw.columns = COLUMNS
11
12 for c in ['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'ca', 'thal']:

```

```

13     raw[c] = pd.to_numeric(raw[c], errors='coerce')
14
15 raw['target'] = (raw['target'] > 0).astype(int)
16 print('Shape:', raw.shape)
17 display(raw.head())
18 display(raw.isna().sum())

```

Tiến hành chia dữ liệu thành các tập train, validation (val) và test.

Chia tập dữ liệu

```

1 TARGET = 'target'
2 raw_feature_cols = [c for c in raw.columns if c != TARGET]
3 X_all = raw[raw_feature_cols]
4 y_all = raw[TARGET]
5
6 X_train, X_temp, y_train, y_temp = train_test_split(
7     X_all, y_all, test_size=0.2, stratify=y_all, random_state=42
8 )
9 X_val, X_test, y_val, y_test = train_test_split(
10     X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42
11 )

```

Một quy trình pipeline được xây dựng để tự động hóa việc tiền xử lý: xử lý các giá trị thiếu và chuẩn hóa dữ liệu, để đảm bảo tính nhất quán giữa các tập. Các tập dữ liệu đã được xử lý này sau đó được lưu thành các tập CSV riêng biệt, tương ứng với train/val/test.

Xây dựng bộ dữ liệu gốc (đã được xử lý)

```

1 cat_proc = Pipeline(steps=[
2     ('imputer', SimpleImputer(strategy='most_frequent')),
3     ('scaler', MinMaxScaler())
4 ])
5 num_proc = Pipeline(steps=[
6     ('imputer', SimpleImputer(strategy='median')),
7     ('scaler', StandardScaler())
8 ])
9
10 preprocess = ColumnTransformer([
11     ('num', num_proc, numeric_cols),
12     ('cat', cat_proc, categorical_cols),
13 ])
14 raw_pipeline = Pipeline([
15     ('preprocess', preprocess),
16 ])
17
18 X_raw_train = raw_pipeline.fit_transform(X_train, y_train)
19 X_raw_val = raw_pipeline.transform(X_val)
20 X_raw_test = raw_pipeline.transform(X_test)
21

```

```

22 preprocessed_feature_names = []
23 for name, transformer, columns in preprocess.transformers_:
24     if hasattr(transformer, 'get_feature_names_out'):
25         preprocessed_feature_names.extend(transformer.get_feature_names_out(columns))
26     else:
27         preprocessed_feature_names.extend(columns)
28
29 X_raw_train_df = pd.DataFrame(
30     X_raw_train, columns=preprocessed_feature_names, index=X_train.index)
31 X_raw_val_df = pd.DataFrame(
32     X_raw_val, columns=preprocessed_feature_names, index=X_val.index)
33 X_raw_test_df = pd.DataFrame(
34     X_raw_test, columns=preprocessed_feature_names, index=X_test.index)
35
36 out_dir = Path('splits'); out_dir.mkdir(parents=True, exist_ok=True)
37 pd.concat([X_raw_train_df, y_train.rename(TARGET)],
38           axis=1).to_csv(out_dir / 'raw_train.csv', index=False)
39 pd.concat([X_raw_val_df, y_val.rename(TARGET)],
40           axis=1).to_csv(out_dir / 'raw_val.csv', index=False)
41 pd.concat([X_raw_test_df, y_test.rename(TARGET)],
42           axis=1).to_csv(out_dir / 'raw_test.csv', index=False)
43
44 display(X_raw_train_df)

```

Sử dụng mô hình Decision Tree để chọn lọc các đặc trưng quan trọng nhất. Độ quan trọng của mỗi đặc trưng được tính toán và sắp xếp, từ đó chọn ra các đặc trưng hàng đầu để tạo ra một bộ dữ liệu mới, tối ưu hóa cho mô hình.

Tạo bộ dữ liệu mới với Decision Tree

```

1 dt_feature_selection_pipeline = Pipeline([
2     ('preprocess', preprocess),
3     ('decision_tree', DecisionTreeClassifier(random_state=42))
4 ])
5
6 dt_feature_selection_pipeline.fit(X_train, y_train)
7 feature_importance_series = pd.Series(
8     dt_feature_selection_pipeline.named_steps['decision_tree'].feature_importances_,
9     index=preprocessed_feature_names
10 )
11 sorted_feature_importances = feature_importance_series.sort_values(ascending=False)
12 display(sorted_feature_importances)
13
14 selected_features = sorted_feature_importances.head(K_features).index.tolist()
15 print(f'Top {K_features} selected features: {selected_features}')
16
17 X_dt_train = X_raw_train_df[selected_features]
18 X_dt_val = X_raw_val_df[selected_features]
19 X_dt_test = X_raw_test_df[selected_features]
20 display(X_dt_train.head())
21
22 pd.concat([X_dt_train, y_train.rename(TARGET)],

```



```

23     axis=1).to_csv(out_dir / 'dt_train.csv', index=False)
24 pd.concat([X_dt_val, y_val.rename(TARGET)],
25           axis=1).to_csv(out_dir / 'dt_val.csv', index=False)
26 pd.concat([X_dt_test, y_test.rename(TARGET)],
27           axis=1).to_csv(out_dir / 'dt_test.csv', index=False)

```



Hình 3: Mức độ ảnh hưởng của các đặc trưng dựa vào Decision Tree trên bộ `dt_train`.

Tiếp tục quá trình bằng cách tạo các đặc trưng mới (Feature Engineering), như tỷ lệ cholesterol theo tuổi, để làm giàu dữ liệu. Các đặc trưng này được tích hợp vào một pipeline riêng, để áp dụng lên bộ dữ liệu gốc (chưa được xử lý).

```

1 def add_new_features_func(df):
2     df = df.copy()
3     if {'chol', 'age'} <= set(df.columns):
4         df['chol_per_age'] = df['chol']/df['age']
5     if {'trestbps', 'age'} <= set(df.columns):
6         df['bps_per_age'] = df['trestbps']/df['age']
7     if {'thalach', 'age'} <= set(df.columns):
8         df['hr_ratio'] = df['thalach']/df['age']
9     if 'age' in df.columns:
10        df['age_bin'] = pd.cut(
11            df['age'], bins=5, labels=False
12        ).astype('category')
13    return df
14

```

```

15 class AddNewFeaturesTransformer(BaseEstimator, TransformerMixin):
16     def __init__(self):
17         pass
18
19     def fit(self, X, y=None):
20         self.columns_ = X.columns
21         self.new_features_ = []
22         if {'chol', 'age'} <= set(X.columns):
23             self.new_features_.append('chol_per_age')
24         if {'trestbps', 'age'} <= set(X.columns):
25             self.new_features_.append('bps_per_age')
26         if {'thalach', 'age'} <= set(X.columns):
27             self.new_features_.append('hr_ratio')
28         if 'age' in X.columns:
29             self.new_features_.append('age_bin')
30         return self
31
32     def transform(self, X):
33         return add_new_features_func(X)
34
35     def get_feature_names_out(self, input_features=None):
36         return list(self.columns_) + self.new_features_
37
38
39 gen_num = ['chol_per_age', 'bps_per_age', 'hr_ratio']
40 gen_cat = ['age_bin']
41 all_nums = [c for c in numeric_cols] + gen_num
42 all_cats = [c for c in categorical_cols] + gen_cat
43
44 num_proc = Pipeline([('imp', SimpleImputer(strategy='median')),
45                      ('sc', StandardScaler())])
46 cat_proc = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
47                      ('ohe', OneHotEncoder(handle_unknown='ignore', sparse_output=False))])
48
49 pre = ColumnTransformer([
50     ('num', num_proc, all_nums),
51     ('cat', cat_proc, all_cats),
52 ], verbose_feature_names_out=False).set_output(transform='pandas')
53 fe_pre = Pipeline([
54     ('add', AddNewFeaturesTransformer()),
55     ('pre', pre),
56 ]).set_output(transform='pandas')
57
58 Xt_tr = fe_pre.fit_transform(X_train, y_train)
59 Xt_va = fe_pre.transform(X_val)
60 Xt_te = fe_pre.transform(X_test)
61
62 nz_cols = Xt_tr.columns[Xt_tr.nunique(dropna=False) > 1]
63 Xt_tr = Xt_tr[nz_cols]
64 Xt_va = Xt_va[nz_cols]
65 Xt_te = Xt_te[nz_cols]

```

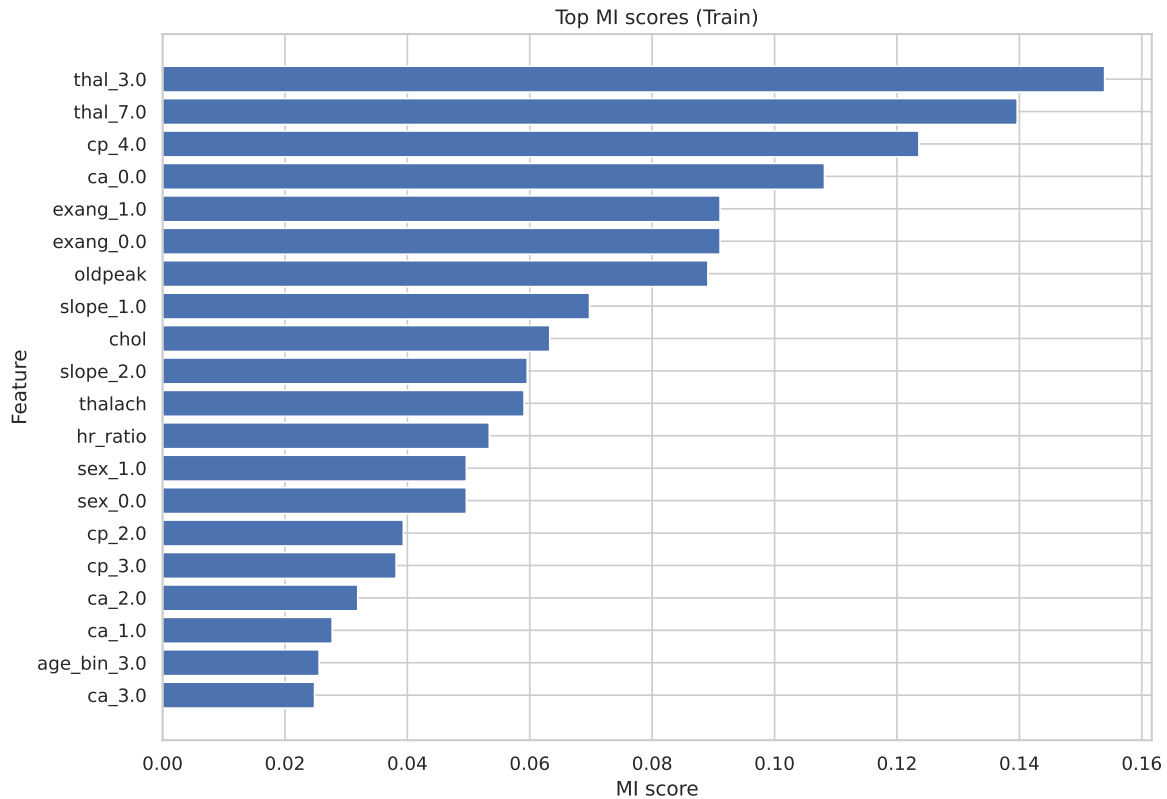
Áp dụng phương pháp Mutual Information (như dự án trước) để đánh giá và chọn lọc các đặc

trung có mối liên hệ cao nhất với biến mục tiêu. Sau đó, lưu lại bộ dữ liệu FE mới được tạo này.

```

1 ohe = fe_pre.named_steps['pre'].named_transformers_['cat'].named_steps['ohe']
2 cat_names = list(ohe.get_feature_names_out(all_cats))
3 is_discrete = np.array(
4     [c in cat_names for c in Xt_tr.columns],
5     dtype=bool
6 )
7 mi = mutual_info_classif(Xt_tr.values, y_train.values,
8                          discrete_features=is_discrete,
9                          random_state=42)
10 mi_series = pd.Series(
11     mi, index=Xt_tr.columns).sort_values(ascending=False)
12
13 N = min(20, len(mi_series))
14 topN = mi_series.head(N).iloc[::-1]
15 plt.figure(figsize=(10, max(6, 0.35*N)))
16 plt.barh(topN.index, topN.values)
17 plt.title('Top MI scores (Train)')
18 plt.xlabel('MI score')
19 plt.ylabel('Feature')
20 plt.tight_layout()
21 plt.savefig('top_mi_scores.pdf', bbox_inches='tight')
22 plt.show()
23
24 K = raw.columns.drop('target').shape[0]
25 topk_cols = list(mi_series.head(K).index)
26 fe_tr = Xt_tr[topk_cols].assign(target=y_train.values)
27 fe_va = Xt_va[topk_cols].assign(target=y_val.values)
28 fe_te = Xt_te[topk_cols].assign(target=y_test.values)
29
30 out = Path('splits'); out.mkdir(parents=True, exist_ok=True)
31 fe_tr.to_csv(out/'fe_train.csv', index=False)
32 fe_va.to_csv(out/'fe_val.csv', index=False)
33 fe_te.to_csv(out/'fe_test.csv', index=False)
34
35 display(pd.Series(
36     topk_cols, name='fe_topk_features'
37 ).reset_index(drop=True))

```



Hình 4: Mức độ ảnh hưởng của các đặc trưng theo phương pháp Mutual Information trên bộ train của tập gốc (đã được xử lý).

Cuối cùng, một lần nữa sử dụng mô hình Decision Tree để chọn lọc đặc trưng từ bộ dữ liệu đã qua kỹ thuật tạo đặc trưng (FE), xác định các đặc trưng có tác động lớn nhất. Từ đó, tạo ra các tập dữ liệu cuối cùng với các đặc trưng này.

Tạo tập dữ liệu mới trên tập FE với Decision Tree

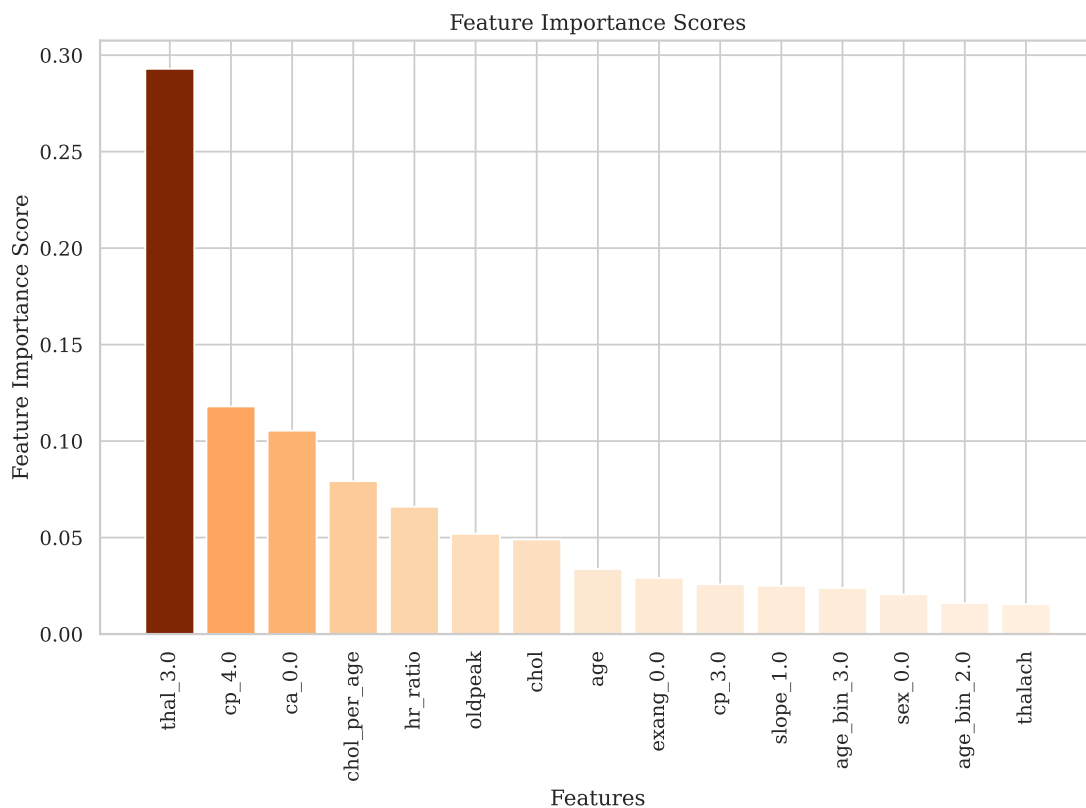
```

1 dt_fe_feature_selection_pipeline = Pipeline([
2     ('preprocess', fe_pre),
3     ('decision_tree', DecisionTreeClassifier(random_state=42))
4 ])
5
6 dt_fe_feature_selection_pipeline.fit(X_train, y_train)
7 pipeline_feature_names = dt_fe_feature_selection_pipeline.named_steps['preprocess'].
8     get_feature_names_out()
9 feature_importance_series = pd.Series(
10     dt_fe_feature_selection_pipeline.named_steps['decision_tree'].feature_importances_,
11     index=pipeline_feature_names
12 )
13 sorted_feature_importances = feature_importance_series.sort_values(ascending=False)
14 selected_features = sorted_feature_importances.head(K_features).index.tolist()
15 print(f'Top {K_features} selected features: {selected_features}')
16
17 X_fe_dt_train = X_tr[selected_features]
```

```

18 X_fe_dt_val = Xt_va[selected_features]
19 X_fe_dt_test = Xt_te[selected_features]
20 display(X_fe_dt_train.head())
21
22 pd.concat([X_fe_dt_train, y_train.rename(TARGET)],
23           axis=1).to_csv(out_dir / 'fe_dt_train.csv', index=False)
24 pd.concat([X_fe_dt_val, y_val.rename(TARGET)],
25           axis=1).to_csv(out_dir / 'fe_dt_val.csv', index=False)
26 pd.concat([X_fe_dt_test, y_test.rename(TARGET)],
27           axis=1).to_csv(out_dir / 'fe_dt_test.csv', index=False)

```



Hình 5: Mức độ ảnh hưởng của các đặc trưng dựa vào Decision Tree trên bộ **train** của tập FE.

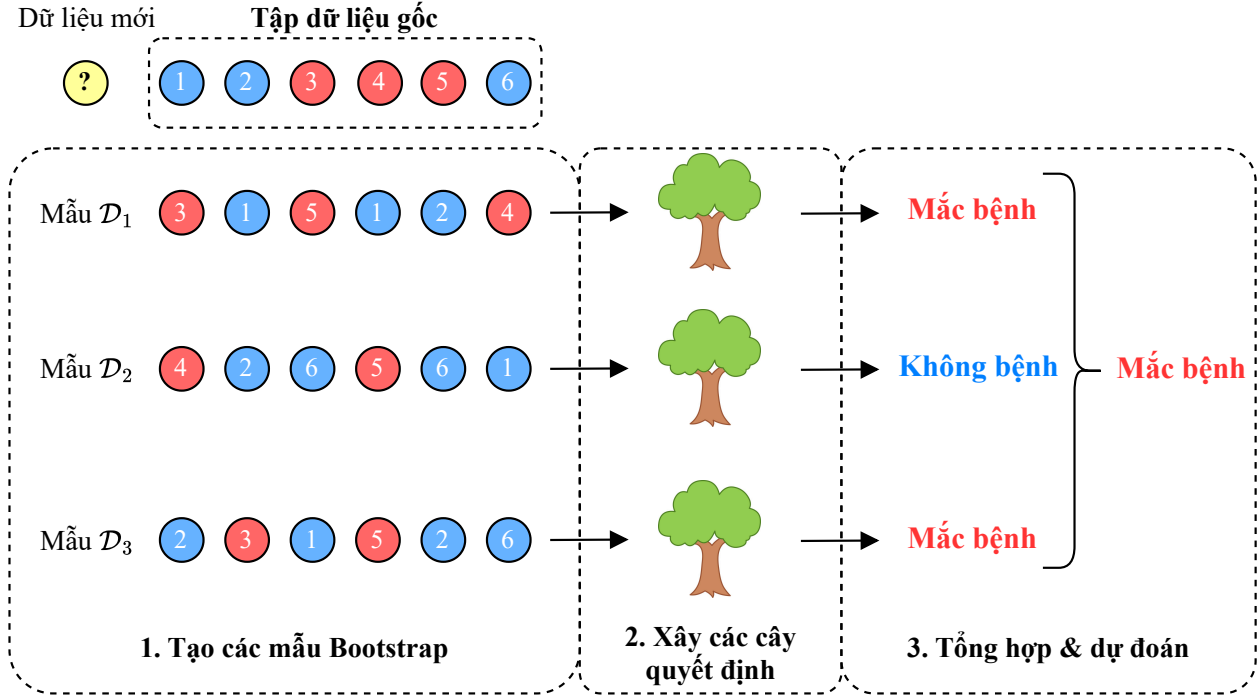
Sử dụng mã bên dưới để nén các tập dữ liệu thành `data.zip` để tải xuống từ Google Colab.

Nén các tập dữ liệu để tải xuống

```
1 !zip -r dataset.zip splits
```

II.3. Random Forest

II.3.1. Giới thiệu



Hình 6: Minh họa cách hoạt động của Random Forest.

Random Forest là một thuật toán học máy có giám sát thuộc nhóm ensemble, hoạt động bằng cách xây dựng một “rừng” gồm nhiều Decision Tree trong quá trình huấn luyện. Mục tiêu chính của thuật toán là cải thiện độ chính xác và khắc phục nhược điểm lớn của một cây quyết định đơn lẻ: xu hướng bị overfitting (quá khớp) và có phương sai cao.

Để đảm bảo các cây trong rừng là khác biệt và không bị tương quan cao với nhau, Random Forest áp dụng hai kỹ thuật ngẫu nhiên hóa cốt lõi:

- Bagging (Bootstrap Aggregating):** Mỗi cây được huấn luyện trên một tập dữ liệu con khác nhau, được lấy ngẫu nhiên có hoàn lại từ tập dữ liệu huấn luyện ban đầu.
- Feature Randomness:** Tại mỗi bước phân nhánh của một cây, thay vì xem xét toàn bộ các đặc trưng, thuật toán chỉ chọn một tập con ngẫu nhiên của các đặc trưng để tìm ra điểm chia tối ưu.

Khi dự đoán trên dữ liệu mới, kết quả cuối cùng được tổng hợp từ tất cả các cây. Cụ thể, mô hình sẽ lấy lớp có nhiều phiếu bầu nhất cho bài toán **phân loại** (cơ chế *majority voting*), hoặc tính giá trị trung bình của các dự đoán cho bài toán **hồi quy**. Bằng cách tổng hợp từ nhiều cây đa dạng, Random Forest tạo ra một mô hình tổng thể vừa mạnh mẽ, chính xác, vừa có khả năng khái quát hóa tốt trên dữ liệu chưa từng thấy.

II.3.2. Triển khai

Lưu ý: Các khối code được đánh dấu “*” nghĩa là được sử dụng nhiều lần mà không cần sửa đổi giữa các mô hình trong dự án này.

Bắt đầu bằng cách nhập các thư viện cần thiết, thiết lập một số ngẫu nhiên chung để đảm bảo tính tái lập của kết quả.

*Tải các thư viện cần thiết

```

1 import os
2 import random
3 import warnings
4 import numpy as np
5 import pandas as pd
6 import seaborn as sns
7 import matplotlib.pyplot as plt
8
9 from xgboost import XGBClassifier
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.ensemble import AdaBoostClassifier
12 from sklearn.ensemble import RandomForestClassifier
13 from sklearn.ensemble import GradientBoostingClassifier
14 from sklearn.metrics import accuracy_score, classification_report
15 from sklearn.model_selection import StratifiedKFold, cross_val_score
16
17 warnings.filterwarnings('ignore')
18
19 SEED = 42
20 os.environ['PYTHONHASHSEED'] = str(SEED)
21 np.random.seed(SEED)
22 random.seed(SEED)
23 print(f'Seed: {SEED}')
```

Sau đó, tải xuống bộ dữ liệu được lưu trữ trên Google Drive.

*Tải các bộ dữ liệu

```

1 # https://drive.google.com/drive/folders/1cMoqIDeGgYDVzv8B7cKp3csxujQ40Fp7?usp=
   drive_link
2 !gdown --folder 1cMoqIDeGgYDVzv8B7cKp3csxujQ40Fp7
```

Tiếp theo, một hàm được tạo để đọc các tệp .csv, hiển thị năm dòng đầu tiên, đếm số lượng các nhãn mục tiêu và in kích thước của dữ liệu.

*Hàm đọc dữ liệu trong file .csv

```

1 def read_csv(file_path):
2     df = pd.read_csv(file_path)
3     display(df.head())
4
```

```

5     X = df.drop('target', axis=1)
6     y = df['target']
7     display(y.value_counts())
8
9     print('Shape df: ', df.shape)
10    print('Shape X: ', X.shape)
11    print('Shape y: ', y.shape)
12
13    return X, y

```

Sử dụng hàm vừa định nghĩa, tiến hành đọc và tải các tập dữ liệu huấn luyện, bao gồm các phiên bản dữ liệu gốc, dữ liệu đã qua xử lý kỹ thuật xử lý đặc trưng (FE) và các tập dữ liệu được tạo bằng cách chọn lọc đặc trưng bằng Decision Tree (DT) trên 2 bộ trên.

*Đọc các tập dữ liệu

```

1  # Original Dataset
2  X_train, y_train = read_csv('dataset_v3/raw_train.csv')
3  X_val, y_val = read_csv('dataset_v3/raw_val.csv')
4  X_test, y_test = read_csv('dataset_v3/raw_test.csv')
5
6  # FE Dataset
7  X_fe_train, y_fe_train = read_csv('dataset_v3/fe_train.csv')
8  X_fe_val, y_fe_val = read_csv('dataset_v3/fe_val.csv')
9  X_fe_test, y_fe_test = read_csv('dataset_v3/fe_test.csv')
10
11 # Original + DT Dataset
12 X_dt_train, y_dt_train = read_csv('dataset_v3/dt_train.csv')
13 X_dt_val, y_dt_val = read_csv('dataset_v3/dt_val.csv')
14 X_dt_test, y_dt_test = read_csv('dataset_v3/dt_test.csv')
15
16 # FE + DT Dataset
17 X_fe_dt_train, y_fe_dt_train = read_csv('dataset_v3/fe_dt_train.csv')
18 X_fe_dt_val, y_fe_dt_val = read_csv('dataset_v3/fe_dt_val.csv')
19 X_fe_dt_test, y_fe_dt_test = read_csv('dataset_v3/fe_dt_test.csv')

```

Sau đó, định nghĩa một hàm để tìm số lượng cây tối ưu cho mô hình Random Forest bằng phương pháp Stratified K-Fold Cross-Validation. Quá trình này giúp đánh giá hiệu suất mô hình trên các giá trị `n_estimators` (số cây con) khác nhau để chọn ra giá trị tốt nhất.

Hàm tìm số lượng cây con tối ưu

```

1  def find_optimal_rf(
2      X_train, y_train, n_estimators_range=range(50, 501, 50), cv_splits=3,
3      max_depth=5, min_samples_split=2, min_samples_leaf=1,
4      max_features='sqrt', bootstrap=True, class_weight=None
5  ):
6      cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
7      scores = []
8      for n in n_estimators_range:

```



```

9         rf = RandomForestClassifier(
10             n_estimators=n,max_depth=max_depth,min_samples_split=min_samples_split,
11             min_samples_leaf=min_samples_leaf,max_features=max_features,
12             bootstrap=bootstrap,class_weight=class_weight,n_jobs=-1,random_state=SEED
13         )
14         cv_score = cross_val_score(rf,X_train,y_train,
15             cv=cv,scoring='accuracy',n_jobs=-1)
16         scores.append(cv_score.mean())
17     plt.figure(figsize=(10, 6))
18     plt.plot(list(n_estimators_range), scores, 'bo-')
19     plt.title(f'Chọn n_estimators tối ưu cho Random Forest (CV={cv_splits}-fold)')
20     plt.xlabel('n_estimators')
21     plt.ylabel('Cross-Validation Accuracy')
22     plt.grid(True)
23     plt.show()
24
25     best_n = list(n_estimators_range)[int(np.argmax(scores))]
26     print(f'n_estimators tối ưu (CV): {best_n}')
27
28     best_model = RandomForestClassifier(
29         n_estimators=best_n,max_depth=max_depth,min_samples_split=min_samples_split,
30         min_samples_leaf=min_samples_leaf,max_features=max_features,
31         bootstrap=bootstrap,class_weight=class_weight,n_jobs=-1,random_state=SEED
32     )
33     best_model.fit(X_train, y_train)
34     return best_model, best_n, max(scores)

```

Tiếp đến, một hàm khác được tạo để huấn luyện và đánh giá mô hình trên tập dữ liệu huấn luyện và thẩm định, sau đó dự đoán và đánh giá hiệu suất trên tập kiểm thử.

Hàm train/val và hàm test

```

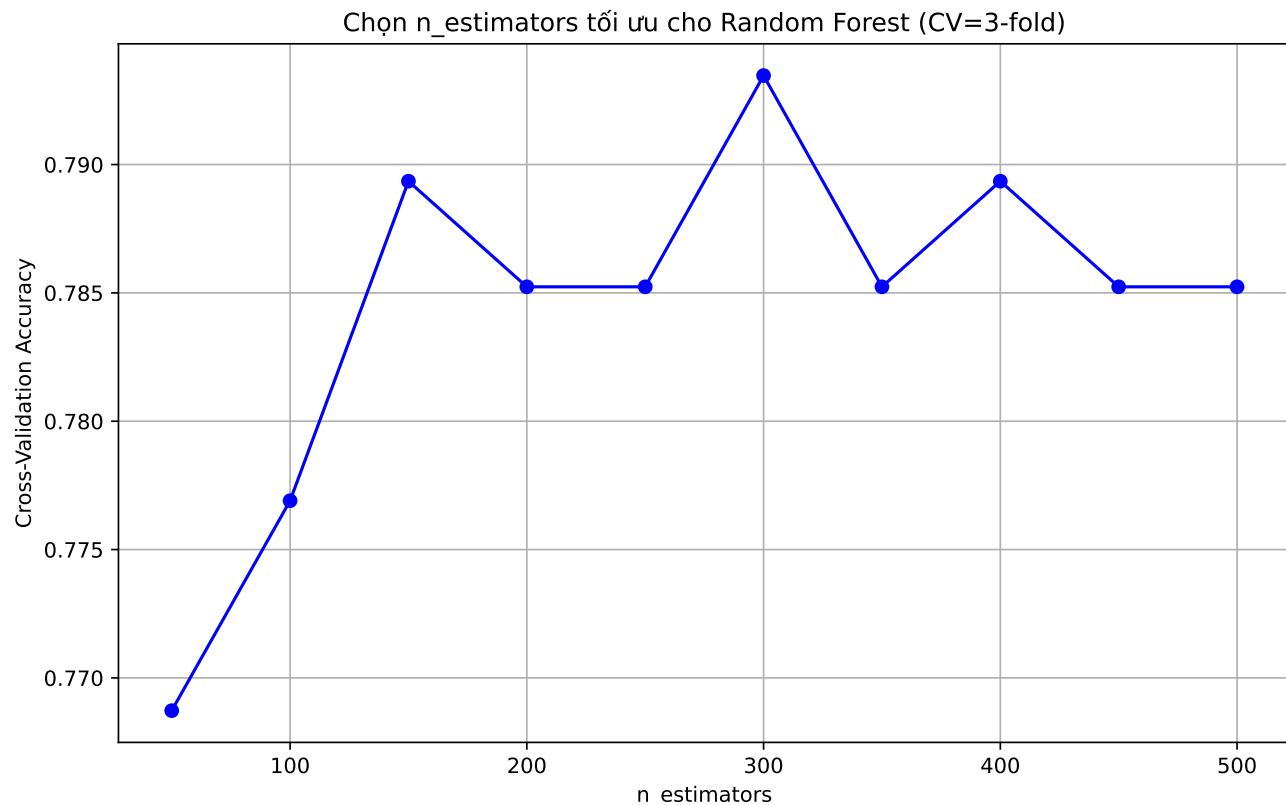
1 def evaluate_val_rf(X_train, y_train, X_val, y_val,
2     n_estimators_range=range(50, 501, 50),cv_splits=5,max_depth=5,
3     min_samples_split=2,min_samples_leaf=1,max_features='sqrt',
4     bootstrap=True,class_weight=None):
5     print('Tìm n_estimators tối ưu cho Random Forest...')
6     rf_model, best_n, cv_acc = find_optimal_rf(
7         X_train,y_train,n_estimators_range=n_estimators_range,cv_splits=cv_splits,
8         max_depth=max_depth,min_samples_split=min_samples_split,
9         min_samples_leaf=min_samples_leaf,max_features=max_features,
10        bootstrap=bootstrap,class_weight=class_weight
11    )
12
13    val_pred = rf_model.predict(X_val)
14    val_acc = accuracy_score(y_val, val_pred)
15    print(f'\nĐộ chính xác Random Forest trên tập validation: {val_acc:.4f}')
16    print('Classification Report:')
17    print(classification_report(y_val, val_pred))
18    return rf_model, val_acc, {'n_estimators': best_n}
19
20 def evaluate_test_rf(rf_model, X_test, y_test):

```

```

21 test_pred = rf_model.predict(X_test)
22 test_acc = accuracy_score(y_test, test_pred)
23 print(f'\nĐộ chính xác Random Forest trên tập test: {test_acc:.4f}')
24 print('Classification Report:')
25 print(classification_report(y_test, test_pred))
26 return test_acc

```



Hình 7: Biểu đồ độ chính xác khi áp dụng Stratified K-Fold Cross-Validation cho mô hình Random Forest tại các giá trị $n_{\text{estimators}}$ khác nhau.

Sử dụng các hàm trên, tiến hành huấn luyện và đánh giá mô hình Random Forest trên bốn bộ dữ liệu khác nhau: dữ liệu gốc, dữ liệu FE, dữ liệu gốc kết hợp với DT, và dữ liệu FE kết hợp với DT.

Huấn luyện và đánh giá trên các tập dữ liệu

```

1 # RF on Original Dataset
2 rf_model, val_acc, best_params = evaluate_val_rf(
3     X_train, y_train, X_val, y_val
4 )
5 test_acc = evaluate_test_rf(rf_model, X_test, y_test)
6
7 # RF on Feature Engineering Dataset

```

```

8 rf_model, val_fe_acc, best_params = evaluate_val_rf(
9     X_fe_train, y_fe_train, X_fe_val, y_fe_val
10 )
11
12 # RF on Original DT Dataset
13 rf_model, val_dt_acc, best_params = evaluate_val_rf(
14     X_dt_train, y_dt_train, X_dt_val, y_dt_val
15 )
16 test_dt_acc = evaluate_test_rf(rf_model, X_dt_test, y_dt_test)
17
18 # RF on Feature Engineering DT Dataset
19 rf_model, val_fe_dt_acc, best_params = evaluate_val_rf(
20     X_fe_dt_train, y_fe_dt_train, X_fe_dt_val, y_fe_dt_val,
21 )
22 test_fe_dt_acc = evaluate_test_rf(rf_model, X_fe_dt_test, y_fe_dt_test)

```

Cuối cùng, một biểu đồ cột được vẽ để so sánh trực quan hiệu suất (độ chính xác) của mô hình trên các tập dữ liệu thẩm định và kiểm thử, giúp dễ dàng nhận biết bộ dữ liệu nào mang lại kết quả tốt nhất.

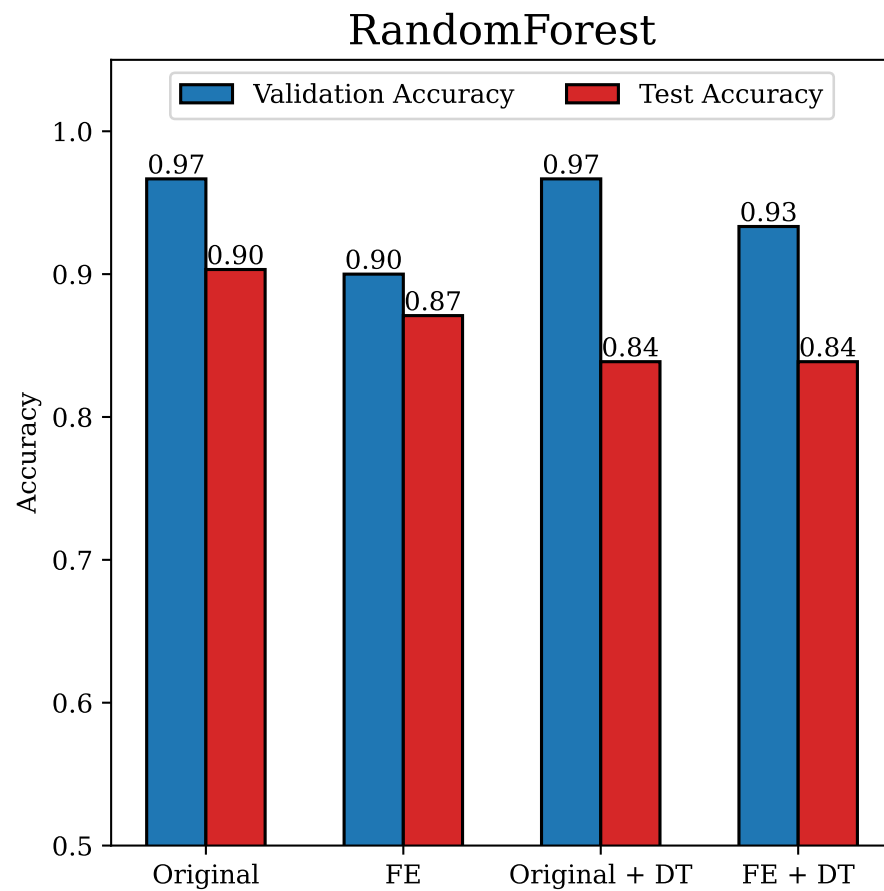
*Vẽ biểu đồ so sánh hiệu suất mô hình trên các tập dữ liệu

```

1 # Just change the title in line 22 to the corresponding model name
2 plt.rcParams['font.family'] = 'Serif'
3
4 labels = ['Original', 'FE', 'Original + DT', "FE + DT"]
5 val_accs = [val_acc, val_fe_acc, val_dt_acc, val_fe_dt_acc]
6 test_accs = [test_acc, test_fe_acc, test_dt_acc, test_fe_dt_acc]
7
8 x = np.arange(len(labels))
9 width = 0.3
10
11 fig, ax = plt.subplots(figsize=(5, 5))
12
13 rects1 = ax.bar(x - width/2, val_accs, width,
14                 label='Validation Accuracy',
15                 color='tab:blue', edgecolor='black', linewidth=1.2)
16 rects2 = ax.bar(x + width/2, test_accs, width,
17                 label='Test Accuracy',
18                 color='tab:red', edgecolor='black', linewidth=1.2)
19
20 ax.set_ylim(0.5, 1.05)
21 ax.set_ylabel('Accuracy')
22 ax.set_title('RandomForest', fontsize=16)
23 ax.set_xticks(x)
24 ax.set_xticklabels(labels)
25 ax.legend(ncol=2, loc="upper center")
26
27 def autolabel(rects):
28     for rect in rects:
29         h = rect.get_height()
30         ax.annotate(f'{h:.2f}', xy=(rect.get_x()+rect.get_width()/2, h),

```

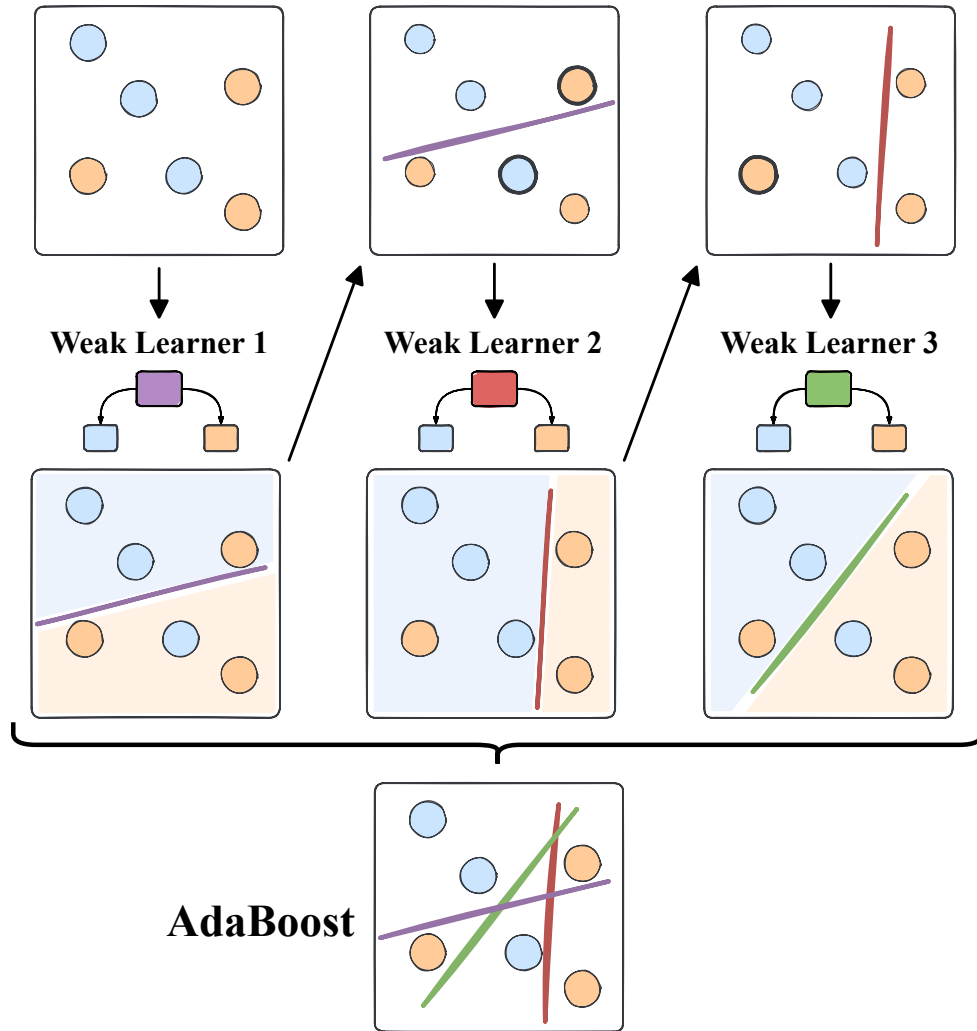
```
31         ha='center', va='bottom')
32
33     autolabel(rects1)
34     autolabel(rects2)
35
36     fig.tight_layout()
37     plt.show()
```



Hình 8: Hiệu suất của mô hình Random Forest trên 4 bộ dữ liệu.

II.4. AdaBoost

II.4.1. Giới thiệu



Hình 9: Hình minh họa thuật toán AdaBoost cho bài toán phân loại.

AdaBoost [2] hoạt động theo một quy trình lặp lại, chú trọng vào các điểm dữ liệu mà các mô hình trước đó đã phân loại sai. Cụ thể như sau:

- Ban đầu, mỗi điểm dữ liệu trong tập huấn luyện đều có một trọng số như nhau.
- Một mô hình yếu (Weak Learner 1), chẳng hạn như một cây quyết định đơn giản (stump), được huấn luyện trên dữ liệu.
- Sau khi huấn luyện mô hình yếu đầu tiên, trọng số của các điểm dữ liệu sẽ được điều chỉnh. Các điểm dữ liệu mà mô hình phân loại sai sẽ được tăng trọng số, trong khi các điểm phân loại đúng sẽ giảm trọng số. Điều này buộc mô hình yếu tiếp theo phải tập trung hơn vào các điểm dữ liệu khó.

- Quá trình này lặp lại nhiều lần với các mô hình yếu mới (Weak Learner 2, Weak Learner 3). Mỗi mô hình mới sẽ được huấn luyện trên tập dữ liệu đã được điều chỉnh trọng số, do đó tập trung vào những lỗi của mô hình trước đó.
- Cuối cùng, AdaBoost sẽ kết hợp tất cả các mô hình yếu lại với nhau thành một mô hình tổng thể. Mỗi mô hình yếu sẽ được gán một trọng số dựa trên hiệu suất của nó - những mô hình có độ chính xác cao sẽ có trọng số lớn hơn trong quyết định cuối cùng.

Kết quả là một mô hình dự đoán mạnh mẽ và có độ chính xác cao, được xây dựng từ sự đóng góp của nhiều mô hình yếu, mỗi mô hình đều tập trung vào việc khắc phục những điểm yếu của mô hình trước đó.

II.4.2. Triển khai

Xây dựng một hàm để tìm số lượng cây con tối ưu cho mô hình AdaBoost bằng cách sử dụng Stratified K-Fold Cross-Validation để đánh giá hiệu suất qua các giá trị `n_estimators` khác nhau.

Hàm tìm số lượng cây con tối ưu

```

1 def find_optimal_ada(
2     X_train, y_train,
3     n_estimators_range=range(50, 501, 50),
4     cv_splits=3,
5     learning_rate=0.1,
6     base_max_depth=1,
7     algorithm='SAMME'
8 ):
9     cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
10    scores = []
11
12    for n in n_estimators_range:
13        ada = AdaBoostClassifier(
14            estimator=DecisionTreeClassifier(max_depth=base_max_depth, random_state=
15                                                SEED),
16            n_estimators=n, learning_rate=learning_rate,
17            algorithm=algorithm, random_state=SEED
18        )
19        cv_score = cross_val_score(
20            ada, X_train, y_train, cv=cv, scoring='accuracy', n_jobs=-1
21        )
22        scores.append(cv_score.mean())
23
24    plt.figure(figsize=(10, 6))
25    plt.plot(list(n_estimators_range), scores, 'bo-')
26    plt.title(f'Chọn n_estimators tối ưu cho AdaBoost (CV={cv_splits}-fold)')
27    plt.xlabel('n_estimators')
28    plt.ylabel('Cross-Validation Accuracy')
29    plt.grid(True)
30    plt.show()
31
32    best_n = list(n_estimators_range)[int(np.argmax(scores))]
33    print(f'n_estimators tối ưu (CV): {best_n}')
```

```

33
34     best_model = AdaBoostClassifier(
35         estimator=DecisionTreeClassifier(max_depth=base_max_depth, random_state=SEED),
36         n_estimators=best_n, learning_rate=learning_rate,
37         algorithm=algorithm, random_state=SEED
38     )
39     best_model.fit(X_train, y_train)
40     return best_model, best_n, max(scores)

```

Tiếp theo, tạo các hàm để huấn luyện và đánh giá mô hình AdaBoost. Một hàm sẽ huấn luyện mô hình trên tập train và đánh giá trên tập val, sau đó hàm còn lại sẽ đánh giá hiệu suất trên tập test.

Hàm train/val và hàm test

```

1  def evaluate_val_ada(X_train, y_train, X_val, y_val,
2                      n_estimators_range=range(50, 501, 50),
3                      cv_splits=3,
4                      learning_rate=0.1,
5                      base_max_depth=1,
6                      algorithm='SAMME'):
7      print('Tìm n_estimators tối ưu cho AdaBoost...')
8      ada_model, best_n, cv_acc = find_optimal_ada(
9          X_train, y_train,
10         n_estimators_range=n_estimators_range,
11         cv_splits=cv_splits,
12         learning_rate=learning_rate,
13         base_max_depth=base_max_depth,
14         algorithm=algorithm
15     )
16
17     val_pred = ada_model.predict(X_val)
18     val_acc = accuracy_score(y_val, val_pred)
19     print(f'\nĐộ chính xác AdaBoost trên tập validation: {val_acc:.4f}')
20     print('Classification Report:')
21     print(classification_report(y_val, val_pred))
22     return ada_model, val_acc, {'n_estimators': best_n}
23
24  def evaluate_test_ada(ada_model, X_test, y_test):
25     test_pred = ada_model.predict(X_test)
26     test_acc = accuracy_score(y_test, test_pred)
27     print(f'\nĐộ chính xác AdaBoost trên tập test: {test_acc:.4f}')
28     print('Classification Report:')
29     print(classification_report(y_test, test_pred))
30     return test_acc

```

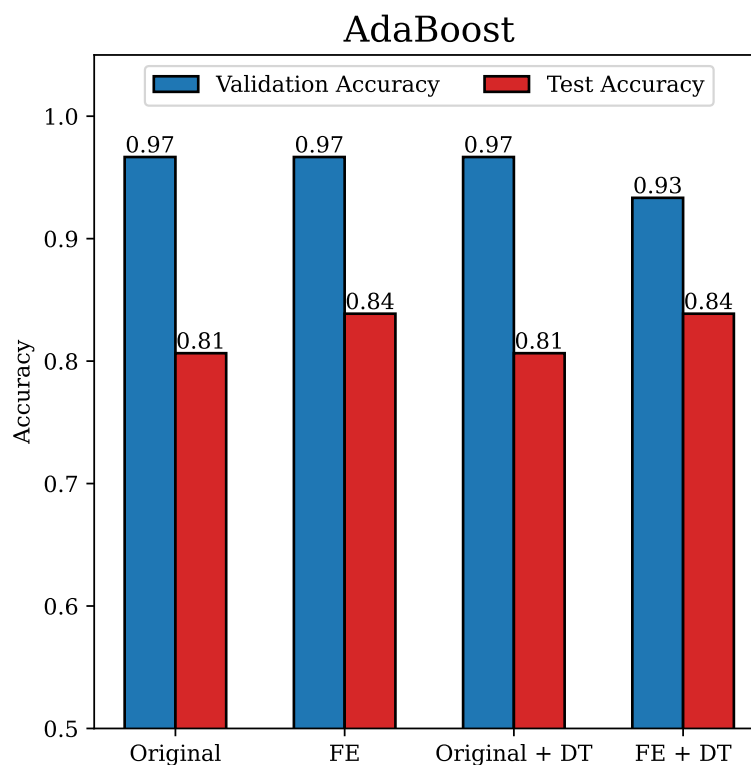
Cuối cùng, sử dụng các hàm đã định nghĩa để huấn luyện và đánh giá mô hình trên bốn bộ dữ liệu khác nhau, bao gồm dữ liệu gốc, dữ liệu đã qua kỹ thuật đặc trưng (FE), dữ liệu kết hợp với Decision Tree (DT), và dữ liệu FE kết hợp với DT.

Huấn luyện và đánh giá trên các tập dữ liệu

```

1  # AB on Original Dataset
2  ada_model, val_acc, best_params = evaluate_val_ada(
3      X_train, y_train, X_val, y_val
4  )
5  test_acc = evaluate_test_ada(ada_model, X_test, y_test)
6
7  # AB on FE Dataset
8  ada_model, val_fe_acc, best_params = evaluate_val_ada(
9      X_fe_train, y_fe_train, X_fe_val, y_fe_val
10 )
11 test_fe_acc = evaluate_test_ada(ada_model, X_fe_test, y_fe_test)
12
13 # AB on Original DT Dataset
14 ada_model, val_dt_acc, best_params = evaluate_val_ada(
15     X_dt_train, y_dt_train, X_dt_val, y_dt_val
16 )
17 test_dt_acc = evaluate_test_ada(ada_model, X_dt_test, y_dt_test)
18
19 # AB on FE + DT Dataset
20 ada_model, val_fe_dt_acc, best_params = evaluate_val_ada(
21     X_fe_dt_train, y_fe_dt_train, X_fe_dt_val, y_fe_dt_val,
22 )
23 test_fe_dt_acc = evaluate_test_ada(ada_model, X_fe_dt_test, y_fe_dt_test)

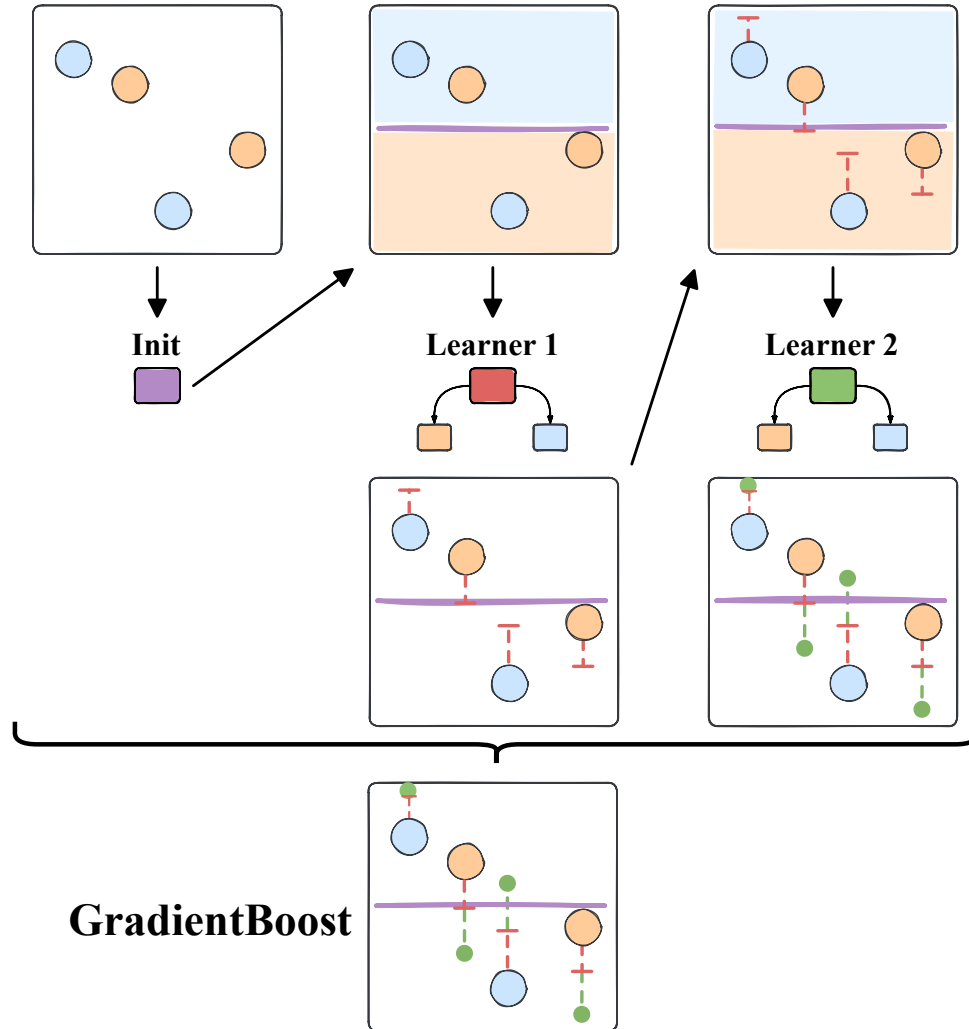
```



Hình 10: Hiệu suất của mô hình AdaBoost trên 4 bộ dữ liệu.

II.5. Gradient Boosting

II.5.1. Giới thiệu



Hình 11: Hình minh họa thuật toán Gradient Boosting cho bài toán phân loại.

Gradient Boosting bắt đầu với một mô hình ban đầu rất đơn giản (Init), thường là một giá trị trung bình hoặc một hằng số. Sau đó lặp lại quá trình sau:

- Tại mỗi bước, thuật toán sẽ tính toán phần dư, tức là sự khác biệt giữa giá trị thực tế và giá trị dự đoán của mô hình hiện tại. Phần dư này chính là "lỗi" mà mô hình cần phải học cách sửa.
- Một mô hình mới, thường là một cây quyết định (Learner 1, Learner 2, ...), được huấn luyện để dự đoán chính xác những phần dư đó. Thay vì dự đoán kết quả cuối cùng, mô hình này tập trung vào việc sửa lỗi của mô hình trước.
- Mô hình mới này sẽ được thêm vào mô hình tổng thể, cùng với một hệ số học (learning rate) nhỏ để kiểm soát tốc độ học, giúp tránh việc mô hình trở nên quá phức tạp (overfitting) một cách nhanh chóng.

Quá trình này lặp lại cho đến khi mô hình tổng thể đạt được độ chính xác mong muốn hoặc đã xây dựng đủ số lượng cây. Kết quả cuối cùng là một mô hình tổng hợp, cực kỳ hiệu quả, được tạo thành từ tổng hợp của tất cả các mô hình nhỏ đã được huấn luyện tuần tự.

II.5.2. Triển khai

Tương tự thuật toán trước, tạo một hàm để tìm số lượng cây con tối ưu cho mô hình Gradient Boosting bằng cách sử dụng Stratified K-Fold Cross-Validation nhằm xác định hiệu suất qua các giá trị `n_estimators`.

Hàm tìm số lượng cây con tối ưu

```

1 def find_optimal_gb(
2     X_train, y_train,
3     n_estimators_range=range(50, 501, 50),
4     cv_splits=3
5 ):
6     cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
7     scores = []
8
9     for n in n_estimators_range:
10         gb = GradientBoostingClassifier(
11             n_estimators=n, learning_rate=0.1,
12             max_depth=5, subsample=1.0, random_state=SEED
13         )
14         cv_score = cross_val_score(
15             gb, X_train, y_train,
16             cv=cv, scoring='accuracy', n_jobs=-1
17         )
18         scores.append(cv_score.mean())
19
20     plt.figure(figsize=(10, 6))
21     plt.plot(list(n_estimators_range), scores, 'bo-')
22     plt.title(f'Chọn n_estimators tối ưu cho Gradient Boosting (CV={cv_splits}-fold)')
23     plt.xlabel('n_estimators')
24     plt.ylabel('Cross-Validation Accuracy')
25     plt.grid(True)
26     plt.show()
27
28     best_n = list(n_estimators_range)[int(np.argmax(scores))]
29     print(f'n_estimators tối ưu (CV): {best_n}')
30
31     best_model = GradientBoostingClassifier(
32         n_estimators=best_n,
33         learning_rate=0.1,
34         max_depth=5,
35         subsample=1.0,
36         random_state=SEED
37     )
38     best_model.fit(X_train, y_train)
39     return best_model, best_n, max(scores)

```

Tiếp theo, xây dựng các hàm để huấn luyện trên tập train và đánh giá mô hình Gradient Boosting trên tập val và tập test.

Hàm train/val và hàm test

```

1 def evaluate_val_gb(X_train, y_train, X_val, y_val,
2                     n_estimators_range=range(50, 501, 50),
3                     cv_splits=3):
4     print('Tìm n_estimators tối ưu cho Gradient Boosting...')
5     gb_model, best_n, cv_acc = find_optimal_gb(
6         X_train, y_train,
7         n_estimators_range=n_estimators_range,
8         cv_splits=cv_splits
9     )
10
11     val_pred = gb_model.predict(X_val)
12     val_acc = accuracy_score(y_val, val_pred)
13     print(f'\nĐộ chính xác GB trên tập validation: {val_acc:.4f}')
14     print('Classification Report:')
15     print(classification_report(y_val, val_pred))
16     return gb_model, val_acc, {'n_estimators': best_n}
17
18 def evaluate_test_gb(gb_model, X_test, y_test):
19     test_pred = gb_model.predict(X_test)
20     test_acc = accuracy_score(y_test, test_pred)
21     print(f'\nĐộ chính xác GB trên tập test: {test_acc:.4f}')
22     print('Classification Report:')
23     print(classification_report(y_test, test_pred))
24     return test_acc

```

Sau đó, tiến hành huấn luyện và đánh giá mô hình trên bốn bộ dữ liệu khác nhau.

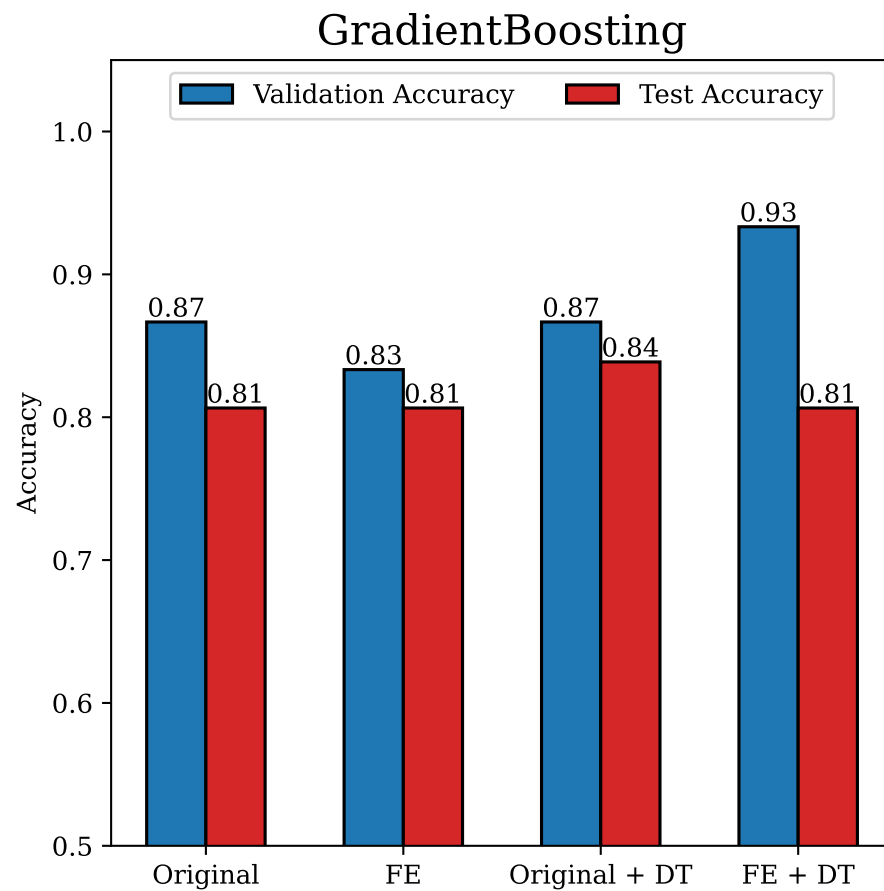
Huấn luyện và đánh giá trên các tập dữ liệu

```

1 # GB on Original Dataset
2 gb_model, val_acc, best_params = evaluate_val_gb(
3     X_train, y_train, X_val, y_val
4 )
5 test_acc = evaluate_test_gb(gb_model, X_test, y_test)
6
7 # GB on Feature Engineering Dataset
8 gb_model, val_fe_acc, best_params = evaluate_val_gb(
9     X_fe_train, y_fe_train, X_fe_val, y_fe_val
10 )
11 test_fe_acc = evaluate_test_gb(gb_model, X_fe_test, y_fe_test)
12
13 # GB on Original DT Dataset
14 gb_model, val_dt_acc, best_params = evaluate_val_gb(
15     X_dt_train, y_dt_train, X_dt_val, y_dt_val
16 )
17 test_dt_acc = evaluate_test_gb(gb_model, X_dt_test, y_dt_test)
18

```

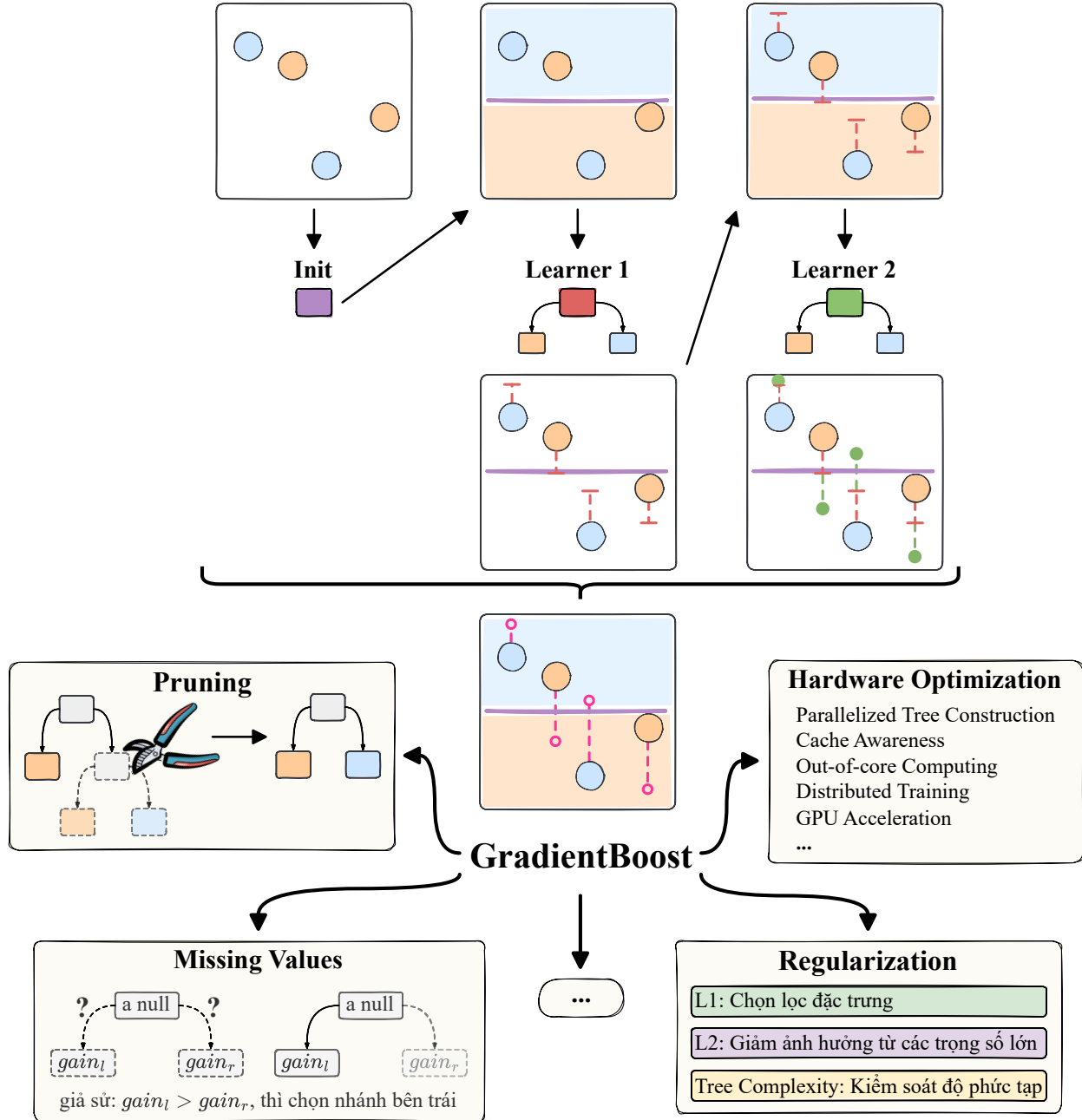
```
19 # GB on Feature Engineering DT Dataset
20 gb_model, val_fe_dt_acc, best_params = evaluate_val_gb(
21     X_fe_dt_train, y_fe_dt_train, X_fe_dt_val, y_fe_dt_val,
22 )
23 test_fe_dt_acc = evaluate_test_gb(gb_model, X_fe_dt_test, y_fe_dt_test)
```



Hình 12: Hiệu suất của mô hình Gradient Boosting trên 4 bộ dữ liệu.

II.6. XGBoost

II.6.1. Giới thiệu



Hình 13: Hình minh họa thuật toán XGBoost cho bài toán phân loại.

XGBoost [3] cải tiến từ Gradient Boosting bằng cách kết hợp nhiều kỹ thuật để tối ưu hóa hiệu suất và giảm thiểu các vấn đề như quá khớp (overfitting). Cụ thể, các đặc điểm nổi bật của nó bao gồm:

- Sử dụng kỹ thuật như Parallelized Tree Construction (xây dựng cây song song) và GPU

Acceleration (tăng tốc bằng GPU) để tăng tốc độ tính toán, đặc biệt với các bộ dữ liệu lớn.

- Tự động xử lý các Missing Values (các giá trị bị thiếu) bằng cách học cách phân nhánh tốt nhất ngay cả khi dữ liệu bị thiếu.
- Áp dụng các kỹ thuật Regularization (điều chuẩn) như L1 và L2 để kiểm soát độ phức tạp của mô hình và tránh việc mô hình học quá sát với dữ liệu huấn luyện. Pruning (cắt tỉa cây) cũng được sử dụng để loại bỏ các nhánh không cần thiết.

Về cơ bản, XGBoost hoạt động bằng cách xây dựng một loạt các cây quyết định một cách tuần tự (như trong sơ đồ là Learner 1 và Learner 2). Mỗi cây mới được thêm vào sẽ cố gắng sửa lỗi của các cây trước đó, dần dần cải thiện độ chính xác tổng thể của mô hình.

II.6.2. Triển khai

Bắt đầu tương tự như các thuật toán trên với một hàm được tìm số lượng cây con tối ưu cho mô hình XGBoost bằng cách sử dụng Stratified K-Fold Cross-Validation với các giá trị `n_estimators` khác nhau.

Hàm tìm số lượng cây con tối ưu

```

1 def find_optimal_xgb(
2     X_train, y_train,
3     n_estimators_range=range(50, 501, 50),
4     cv_splits=3,
5     learning_rate=0.1,
6     max_depth=5,
7     subsample=1.0,
8     use_gpu=False
9 ):
10     cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
11     scores = []
12
13     n_classes = len(np.unique(y_train))
14     objective = 'binary:logistic' if n_classes == 2 else 'multi:softprob'
15     eval_metric = 'logloss' if n_classes == 2 else 'mlogloss'
16
17     for n in n_estimators_range:
18         xgb = XGBClassifier(
19             n_estimators=n, learning_rate=learning_rate, max_depth=max_depth, subsample=
20                 subsample,
21             objective=objective, eval_metric=eval_metric, random_state=SEED, n_jobs=-1,
22             tree_method='gpu_hist' if use_gpu else 'hist', verbosity=0
23         )
24         cv_score = cross_val_score(
25             xgb, X_train, y_train,
26             cv=cv, scoring='accuracy', n_jobs=-1
27         )
28         scores.append(cv_score.mean())
29
30     plt.figure(figsize=(10, 6))
31     plt.plot(list(n_estimators_range), scores, 'bo-')
```

```

31 plt.title(f'Chọn n_estimators tối ưu cho XGBoost (CV={cv_splits}-fold)')
32 plt.xlabel('n_estimators')
33 plt.ylabel('Cross-Validation Accuracy')
34 plt.grid(True)
35 plt.show()
36
37 best_n = list(n_estimators_range)[int(np.argmax(scores))]
38 print(f'n_estimators tối ưu (CV): {best_n}')
39
40 best_model = XGBClassifier(
41     n_estimators=best_n, learning_rate=learning_rate, max_depth=max_depth,
42                                     subsample=subsample,
43     objective=objective, eval_metric=eval_metric, random_state=SEED, n_jobs=-1,
44     tree_method='gpu_hist' if use_gpu else 'hist', verbosity=0
45 )
46 best_model.fit(X_train, y_train)
47 return best_model, best_n, max(scores)

```

Dựa trên hàm tìm kiếm tham số tối ưu, các hàm tiếp theo được tạo để huấn luyện và đánh giá hiệu suất của mô hình.

Hàm train/val và hàm test

```

1 def evaluate_val_xgb(X_train, y_train, X_val, y_val,
2                     n_estimators_range=range(50, 501, 50), cv_splits=3,
3                     learning_rate=0.1, max_depth=5, subsample=1.0,
4                     colsample_bytree=1.0, use_gpu=False):
5     print('Tìm n_estimators tối ưu cho XGBoost...')
6     xgb_model, best_n, cv_acc = find_optimal_xgb(
7         X_train, y_train, n_estimators_range=n_estimators_range,
8         cv_splits=cv_splits, learning_rate=learning_rate,
9         max_depth=max_depth, subsample=subsample, use_gpu=use_gpu
10    )
11
12    val_pred = xgb_model.predict(X_val)
13    val_acc = accuracy_score(y_val, val_pred)
14    print(f'\nĐộ chính xác XGBoost trên tập validation: {val_acc:.4f}')
15    print('Classification Report:')
16    print(classification_report(y_val, val_pred))
17    return xgb_model, val_acc, {'n_estimators': best_n}
18
19 def evaluate_test_xgb(xgb_model, X_test, y_test):
20     test_pred = xgb_model.predict(X_test)
21     test_acc = accuracy_score(y_test, test_pred)
22     print(f'\nĐộ chính xác XGBoost trên tập test: {test_acc:.4f}')
23     print('Classification Report:')
24     print(classification_report(y_test, test_pred))
25     return test_acc

```

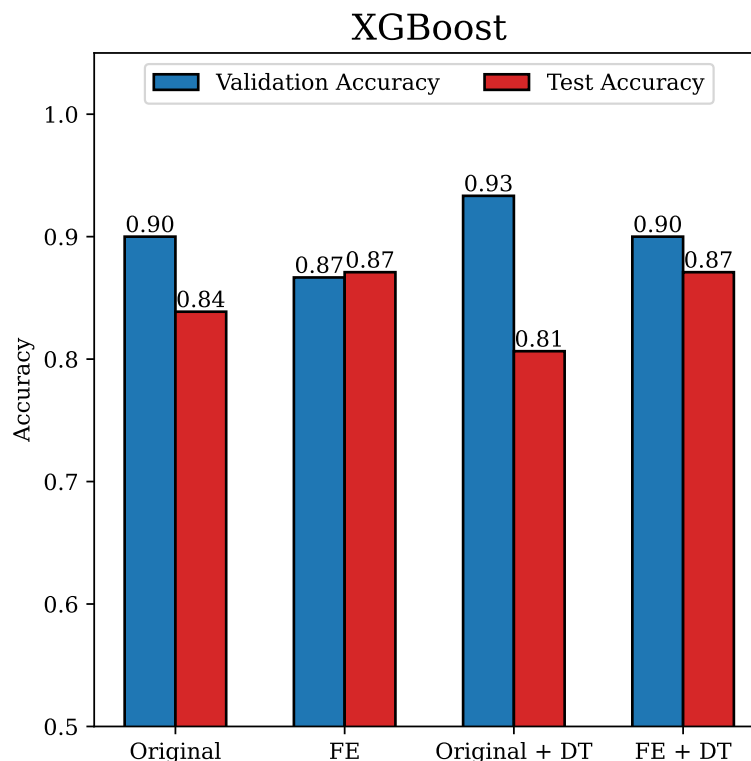
Sau khi đã có các hàm cần thiết, tiến hành huấn luyện và đánh giá mô hình trên bốn bộ dữ liệu khác nhau.

Huấn luyện và đánh giá trên các tập dữ liệu

```

1  # XGB on Original Dataset
2  xgb_model, val_acc, best_params = evaluate_val_xgb(
3      X_train, y_train, X_val, y_val
4  )
5  test_acc = evaluate_test_xgb(xgb_model, X_test, y_test)
6
7  # XGB on Feature Engineering Dataset
8  xgb_model, val_fe_acc, best_params = evaluate_val_xgb(
9      X_fe_train, y_fe_train, X_fe_val, y_fe_val
10 )
11 test_fe_acc = evaluate_test_xgb(xgb_model, X_fe_test, y_fe_test)
12
13 # XGB on Original DT Dataset
14 xgb_model, val_dt_acc, best_params = evaluate_val_xgb(
15     X_dt_train, y_dt_train, X_dt_val, y_dt_val
16 )
17 test_dt_acc = evaluate_test_xgb(xgb_model, X_dt_test, y_dt_test)
18
19 # XGB on Feature Engineering DT Dataset
20 xgb_model, val_fe_dt_acc, best_params = evaluate_val_xgb(
21     X_fe_dt_train, y_fe_dt_train, X_fe_dt_val, y_fe_dt_val,
22 )
23 test_fe_dt_acc = evaluate_test_xgb(xgb_model, X_fe_dt_test, y_fe_dt_test)

```



Hình 14: Hiệu suất của mô hình XGBoost trên 4 bộ dữ liệu.

II.7. Tổng hợp kết quả

Trong phần này, ta lập một bảng ghi nhận độ chính xác của các phương pháp trên tập val và test của từng bộ dữ liệu. Đây là một cách làm thường thấy trong các bài báo khoa học, cung cấp cho người đọc một góc nhìn toàn diện về khả năng của tất cả các phương pháp học máy được xem xét, trong trường hợp này là thuộc xuyên suốt 2 phần của dự án chẩn đoán bệnh tim của chúng ta. Theo đó, các phương pháp được đặt lần lượt theo thứ tự trình bày trong project. Có 4 tập dữ liệu bao gồm bộ gốc (Origin) và các bộ với các kiểu phối hợp tiền xử lý khác nhau như Feature Engineering (FE) và chọn lọc đặc trưng với Decision Tree (DT). Hiệu suất của các mô hình được đo bằng Accuracy (độ chính xác).

Bảng 2: Bảng tổng hợp các kết quả thực nghiệm trên cả 2 dự án (Part 1 và 2). Trong đó, FE là viết tắt của “Feature Engineering”; DT là viết tắt của “Decision Tree”.

Model	Val				Test			
	Origin	FE	Origin+DT	FE+DT	Origin	FE	Origin+DT	FE+DT
Naive Bayes	.90	.90	.93	.93	.84	.84	.84	.84
KNN	.90	.90	.97	.87	.84	.84	.87	.84
KMeans	.70	.80	.83	.63	.87	.87	.84	.77
Decision Tree	.93	.93	.93	.93	.81	.81	.81	.81
Ensemble (KNN,DT,NB)	.87	.93	.90	.87	.84	.90	.84	.84
Random Forest	.97	.90	.97	.93	.90	.87	.84	.84
AdaBoost	.97	.97	.97	.93	.81	.84	.81	.84
Gradient Boosting	.87	.83	.87	.93	.81	.81	.84	.81
XGBoost	.90	.87	.93	.90	.84	.87	.81	.87

Dựa vào Bảng 2, ta nhận thấy độ chính xác cao nhất có thể đạt được trong bài là 97%. Các phương pháp thuộc nhóm Ensemble Learning, cụ thể với Random Forest và AdaBoost, cho kết quả khả quan nhất trên cả 4 kiểu dataset. Bên cạnh đó, việc áp dụng tiền xử lý dữ liệu nhìn chung có thể giúp các phương pháp học máy cải thiện độ chính xác một cách tương đối. Điều này phần nào phản ánh cho chúng ta tầm quan trọng của việc áp dụng tiền xử lý dữ liệu cũng như độ hiệu quả của các phương pháp có ứng dụng Ensemble Learning.

II.8. Cài đặt UI và Deploy

Để nâng cao tính hoàn thiện và tính trực quan của dự án, việc xây dựng một bản demo với giao diện đơn giản là cần thiết. Bản demo đóng vai trò minh họa trực tiếp cho hiệu quả của mô hình, đồng thời hỗ trợ quá trình kiểm thử. Qua đó, kết quả nghiên cứu không chỉ dừng lại ở mức lý thuyết, mà còn được cụ thể hoá thành một sản phẩm có thể sử dụng được.

Google Colab không được thiết kế để lưu trữ ứng dụng web, nên để chạy và truy cập giao diện Streamlit từ Colab ta cần tạo một đường hầm (tunnel) an toàn. Quy trình gồm: (1) viết mã ứng dụng vào app.py, (2) cài các phụ thuộc và khởi chạy Streamlit trên Colab, (3) dùng Cloudflared để mở tunnel từ máy chủ Colab ra ngoài, qua đó nhận một URL công khai để truy cập giao diện từ trình duyệt.

Bước 1: Tạo file ứng dụng app.py

Đầu tiên, chúng ta cần viết toàn bộ code của ứng dụng Streamlit vào một file Python. Trong môi trường Colab, ta sử dụng “magic command” `%%writefile` để thực hiện việc này. Cell đầu tiên này sẽ tạo ra file `app.py` và chứa toàn bộ logic của ứng dụng, từ tải dữ liệu, huấn luyện mô hình cho đến hiển thị giao diện.

Khai báo thư viện và Cấu hình

app.py - Phần 1

```
1 %%writefile app.py
2 import streamlit as st
3 import pandas as pd
4 from sklearn.model_selection import GridSearchCV
5 from sklearn.preprocessing import OneHotEncoder
6 from sklearn.compose import ColumnTransformer
7 from sklearn.pipeline import Pipeline
8 from sklearn.impute import SimpleImputer
9 from sklearn.tree import DecisionTreeClassifier, plot_tree
10 import matplotlib.pyplot as plt
11
12 st.set_page_config(page_title='Heart Disease Prediction', layout='wide')
13 URL = 'https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/
14         processed.cleveland.data'
15 COLUMNS = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
16             'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
17 NUMERIC = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
18 CATEGORICAL = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
```

- **Giải thích:** Import các thư viện cần thiết, bao gồm `GridSearchCV` để tự động tối ưu tham số. Chúng ta cũng cấu hình tiêu đề, icon cho trang web và định nghĩa các hằng số chứa URL dữ liệu và danh sách các cột.

Tải dữ liệu, Tối ưu hóa và Huấn luyện Mô hình

app.py - Phần 2

```
1 @st.cache_resource
2 def find_best_model():
3     df = pd.read_csv(URL, header=None, names=COLUMNS, na_values='?')
4     df = df.dropna()
5     df['target'] = (df['target'] > 0).astype(int)
6
7     X = df[NUMERIC + CATEGORICAL]
8     y = df['target']
9
10    num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median'))])
11    cat_pipe = Pipeline([
12        ('imputer', SimpleImputer(strategy='most_frequent')),
```

```

13     ('ohe', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
14 ]))
15
16     preprocessor = ColumnTransformer([
17         ('num', num_pipe, NUMERIC),
18         ('cat', cat_pipe, CATEGORICAL)
19     ])
20
21     pipeline = Pipeline([
22         ('preprocessor', preprocessor),
23         ('classifier', DecisionTreeClassifier(random_state=42))
24     ])
25
26     param_grid = {'classifier__max_depth': range(3, 11)}
27
28     grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-
                                1)
29     grid_search.fit(X, y)
30
31     return grid_search
32
33 model = find_best_model()

```

- **Giải thích:** Đây là phần logic cốt lõi. Hàm `find_best_model` thực hiện toàn bộ quy trình:
 - Tải dữ liệu từ URL và làm sạch cơ bản.
 - Xây dựng một Pipeline hoàn chỉnh để xử lý dữ liệu số, dữ liệu phân loại và đưa vào mô hình Cây quyết định.
 - Định nghĩa một không gian tham số (`param_grid`) để tìm kiếm độ sâu (`max_depth`) tốt nhất cho cây.
 - Sử dụng `GridSearchCV` để thực hiện kiểm định chéo (cross-validation) 5-fold, tự động tìm ra tham số tối ưu và huấn luyện lại mô hình tốt nhất trên toàn bộ dữ liệu.
 - Decorator `@st.cache_resource` đảm bảo toàn bộ quá trình tốn nhiều thời gian này chỉ chạy một lần duy nhất khi ứng dụng khởi động. Kết quả (mô hình đã được tối ưu) sẽ được lưu vào cache và tái sử dụng, giúp ứng dụng phản hồi ngay lập tức.

Xây dựng Giao diện Người dùng (UI)

app.py - Phần 3

```

1 st.title('Heart Disease Prediction App')
2 st.write('Enter patient data in the sidebar. The model will predict the likelihood of
           heart disease.')
3 st.sidebar.info(f'Optimal model depth found: {model.best_params_['classifier__max_depth
                  ']}')
4
5 with st.sidebar.form('input_form'):
6     st.header('Patient Parameters')
7     age = st.slider('Age', 20, 80, 50)

```

```

8     sex = st.selectbox('Gender', [('Male', 1), ('Female', 0)], format_func=lambda x: x[
        0])[1]
9     cp = st.selectbox('Chest Pain Type', [('Typical Angina', 1), ('Atypical Angina', 2)
        , ('Non-anginal Pain', 3), ('Asymptomatic',
        4)], format_func=lambda x: x[0])[1]
10    trestbps = st.slider('Resting Blood Pressure (mmHg)', 90, 200, 120)
11    chol = st.slider('Serum Cholesterol (mg/dl)', 120, 570, 240)
12    fbs = st.selectbox('Fasting Blood Sugar > 120 mg/dl', [('False', 0), ('True', 1)],
        format_func=lambda x: x[0])[1]
13    restecg = st.selectbox('Resting ECG', [('Normal', 0), ('ST-T Abnormality', 1), ('LV
        Hypertrophy', 2)], format_func=lambda x: x
        [0])[1]
14    thalach = st.slider('Max Heart Rate Achieved', 70, 210, 150)
15    exang = st.selectbox('Exercise-induced Angina', [('No', 0), ('Yes', 1)],
        format_func=lambda x: x[0])[1]
16    oldpeak = st.slider('ST Depression', 0.0, 6.2, 1.0, step=0.1)
17    slope = st.selectbox('Slope of Peak ST', [('Upsloping', 1), ('Flat', 2), ('
        Downsloping', 3)], format_func=lambda x: x[
        0])[1]
18    ca = st.slider('Major Vessels colored by Fluoroscopy', 0, 3, 0)
19    thal = st.selectbox('Thalassemia', [('Normal', 3), ('Fixed Defect', 6), ('
        Reversible Defect', 7)], format_func=lambda
        x: x[0])[1]
20    submitted = st.form_submit_button('Predict')

```

- **Giải thích:** Đoạn code này tạo tiêu đề và các thành phần tương tác. `st.sidebar.form` tạo một form ở thanh bên, gom tất cả các widget nhập liệu lại. Một điểm mới là `st.sidebar.info`, nó sẽ hiển thị độ sâu tối ưu mà GridSearchCV đã tìm được, giúp người dùng biết mô hình đang chạy với cấu hình nào.

Xử lý, Dự đoán và Trực quan hóa

app.py - Phần 4

```

1  if submitted:
2      input_data = pd.DataFrame([{'age': age, 'sex': sex, 'cp': cp, 'trestbps': trestbps,
        'chol': chol, 'fbs': fbs, 'restecg':
        restecg, 'thalach': thalach, 'exang': exang
        , 'oldpeak': oldpeak, 'slope': slope, 'ca':
        ca, 'thal': thal}])
3      proba = model.predict_proba(input_data)[0]
4      prob_disease = proba[1]
5
6      st.subheader('Prediction Result')
7      if prob_disease > 0.5:
8          st.error(f'**Disease Likely** - Probability: {prob_disease:.2%}')
9      else:
10         st.success(f'**Disease Unlikely** - Probability of No Disease: {(1-prob_disease)
            :.2%}')
11
12 st.subheader('Optimal Model Decision Tree Visualization')

```

```

13 st.write('This chart shows the decision rules of the best model found via Cross-
    Validation.')
14
15 best_pipeline = model.best_estimator_
16 preprocessor = best_pipeline.named_steps['preprocessor']
17 classifier = best_pipeline.named_steps['classifier']
18
19 try:
20     feature_names = preprocessor.get_feature_names_out()
21     fig, ax = plt.subplots(figsize=(25, 12))
22     plot_tree(classifier, feature_names=feature_names, class_names=['No Disease', '
        Disease'], filled=True, rounded=True, ax=ax
        , fontsize=10)
23     st.pyplot(fig)
24 except Exception as e:
25     st.warning(f'Could not display the tree visualization. Error: {e}')

```

- **Giải thích:** Khi người dùng nhấn “Predict”, code sẽ thu thập dữ liệu, tạo DataFrame và đưa vào mô hình để dự đoán xác suất. Kết quả được hiển thị trực quan. Phần cuối cùng trực quan hóa cây quyết định. Lưu ý rằng `model` lúc này là một đối tượng `GridSearchCV`, vì vậy chúng ta cần truy cập `model.best_estimator_` để lấy ra pipeline tốt nhất. Từ đó, ta trích xuất bộ phân loại (classifier) và vẽ cây quyết định tương ứng với mô hình đã được tối ưu.

Bước 2: Cài đặt môi trường

Chúng ta cần cài đặt Streamlit và công cụ **Cloudflared** để tạo đường hầm.

Cài đặt Streamlit

```
1 !pip -q install streamlit
```

- **Giải thích:** Lệnh này sử dụng `pip` để cài đặt thư viện `streamlit`.

Tải và cấp quyền cho Cloudflared

```

1 !wget -q https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared
    -linux-amd64 -O cloudflared
2 !chmod +x cloudflared

```

- **Giải thích:** Tải về file thực thi của Cloudflared và cấp quyền để có thể chạy nó.

Bước 3: Khởi chạy ứng dụng

Đây là bước cuối cùng để khởi động server và tạo link truy cập công khai.

Chạy Streamlit Server dưới nền

```
1 !streamlit run app.py \  
2 --server.port 8501 \  
3 --server.address 0.0.0.0 \  
4 --server.headless true \  
5 --server.enableCORS false \  
6 --server.enableXsrfProtection false &>/content/logs.txt &
```

- **Giải thích:** Lệnh này khởi chạy ứng dụng `app.py` trên cổng 8501 và chạy ngầm dưới nền, cho phép chúng ta thực hiện lệnh tiếp theo.

Tạo đường hầm với Cloudflared

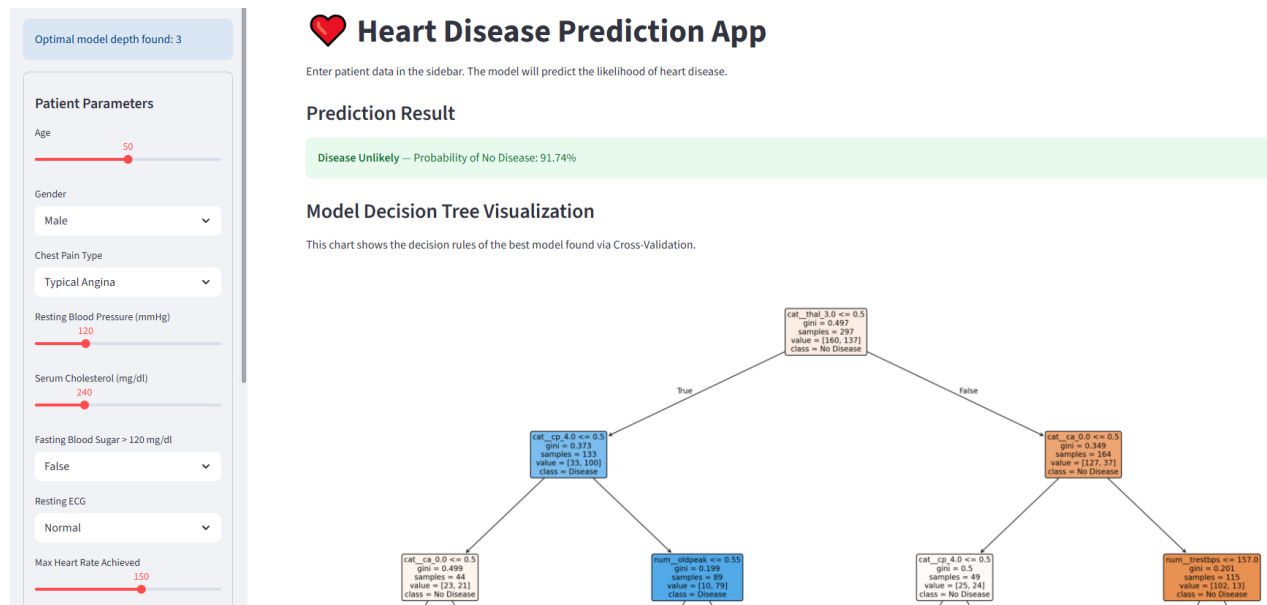
```
1 !./cloudflared tunnel --url http://localhost:8501 --no-autoupdate
```

- **Giải thích:** Lệnh này tạo một đường hầm công khai đến ứng dụng đang chạy cục bộ trên cổng 8501 của server Colab.

Kết quả và Cách truy cập

Sau khi chạy cell cuối cùng, bạn sẽ thấy một loạt các dòng output. Hãy tìm một đường link có dạng **`https://....trycloudflare.com`**. Đây chính là địa chỉ công khai của ứng dụng Streamlit của bạn.

Hãy nhấp vào đường link để mở ứng dụng trong một tab mới!



Hình 15: Giao diện demo.

III. Câu hỏi trắc nghiệm

1. Xét đoạn code chia dữ liệu sau:

```
1 X_train, X_temp, y_train, y_temp = train_test_split(  
2     X_all, y_all, test_size=0.2, stratify=y_all, random_state=42)
```

Tham số `stratify=y_all` có vai trò quan trọng nhất là gì trong bài toán này?

- (a) Đảm bảo các đặc trưng trong tập huấn luyện và kiểm thử có cùng phân phối.
 - (b) Giúp quá trình chia dữ liệu nhanh hơn bằng cách lấy mẫu có thứ tự.
 - (c) Đảm bảo tỷ lệ người có bệnh và không có bệnh được giữ nguyên trong các tập dữ liệu sau khi chia, tránh thiên vị do mất cân bằng nhãn.
 - (d) Xáo trộn dữ liệu ngẫu nhiên để loại bỏ sự phụ thuộc về thứ tự.
2. Cho các pipeline xử lý dữ liệu sau:

```
1 cat_proc = Pipeline(steps=[  
2     ('imputer', SimpleImputer(strategy='most_frequent')),  
3     ('scaler', MinMaxScaler())  
4 ])  
5 num_proc = Pipeline(steps=[  
6     ('imputer', SimpleImputer(strategy='median')),  
7     ('scaler', StandardScaler())  
8 ])  
9
```

Sau khi áp dụng `fit_transform`, một cột phân loại (ví dụ: `sex`) và một cột số (ví dụ: `age`) sẽ có đặc điểm gì?

- (a) Cột `sex` có giá trị trong khoảng `[0, 1]` và cột `age` có trung bình 0, độ lệch chuẩn 1.
 - (b) Cả hai cột đều có trung bình 0 và độ lệch chuẩn 1.
 - (c) Cột `sex` được điền bằng giá trị mode, cột `age` được điền bằng giá trị median, không có scaling.
 - (d) Cả hai cột đều có giá trị trong khoảng `[0, 1]`.
3. So sánh Hình 4 (Mutual Information) và Hình 5 (Decision Tree), sự khác biệt chính trong cách hai phương pháp đánh giá đặc trưng là gì?
- (a) Cả hai đều xếp hạng `thal`, `cp`, `ca` cao nhất, cho thấy chúng hoạt động theo cùng một nguyên lý.
 - (b) Mutual Information đánh giá đặc trưng đã mã hóa one-hot (ví dụ: `thal_3.0`) dựa trên sự phụ thuộc thống kê với nhãn, trong khi Decision Tree đánh giá đặc trưng gốc (ví dụ: `thal`) dựa trên khả năng giảm impurity tại các điểm chia.

- (c) Decision Tree ưu tiên các đặc trưng số như `oldpeak` và `chol` hơn Mutual Information.
 - (d) Mutual Information chỉ áp dụng được trên dữ liệu FE, còn Decision Tree dùng được cho cả dữ liệu gốc.
4. Hàm `find_optimal_rf` cố định `max_depth=5`. Việc giới hạn độ sâu cây trong khi tìm kiếm `n_estimators` có thể gây ra hệ quả tiềm tàng nào?

```

1 def find_optimal_rf(
2     # ...
3     max_depth=5, min_samples_split=2, min_samples_leaf=1,
4     max_features='sqrt', bootstrap=True, class_weight=None
5 ): # ...
6

```

- (a) Mô hình chắc chắn sẽ bị quá khớp (overfitting).
 - (b) Giúp mô hình hội tụ nhanh và luôn tìm được cấu hình tốt nhất toàn cục.
 - (c) Có thể khiến mô hình bị chưa khớp (underfitting) vì mỗi cây không đủ phức tạp để nắm bắt các mối quan hệ phức tạp, ngay cả khi có nhiều cây.
 - (d) Tham số này không ảnh hưởng đến hiệu suất, chỉ có `n_estimators` là quan trọng.
5. Trong đoạn code XGBoost sau, tham số `tree_method` đang áp dụng kỹ thuật tối ưu hóa hiệu năng nào?

```

1 xgb = XGBClassifier(
2     # ...
3     tree_method='gpu_hist' if use_gpu else 'hist', verbosity=0
4 )
5

```

- (a) Kỹ thuật điều chuẩn L2 để kiểm soát trọng số của mô hình.
 - (b) Thuật toán xây dựng cây dựa trên histogram giúp tăng tốc độ tìm điểm chia, và có tùy chọn tăng tốc bằng GPU.
 - (c) Cơ chế tự động xử lý giá trị thiếu mà không cần imputation.
 - (d) Kỹ thuật cắt tỉa cây (pruning) sau khi xây dựng hoàn chỉnh.
6. Trong code Streamlit, biến `model` được dùng theo hai cách. Tại sao cần truy cập `model.best_estimator_` để vẽ cây, trong khi có thể dùng `model` trực tiếp để dự đoán?

```

1 # Cách 1: Dự đoán
2 proba = model.predict_proba(input_data)[0]
3
4 # Cách 2: Lấy thông tin để vẽ cây
5 best_pipeline = model.best_estimator_

```

```

6 classifier = best_pipeline.named_steps['classifier']
7 plot_tree(classifier, ...)
8

```

- (a) Đây là quy ước đặt tên, `model` và `model.best_estimator_` thực chất là một.
 - (b) Lệnh `plot_tree` yêu cầu một pipeline, còn `predict_proba` chỉ cần một mô hình phân loại.
 - (c) Biến `model` là đối tượng `GridSearchCV`. Các hàm như `predict_proba` được ủy quyền cho mô hình tốt nhất, nhưng để truy cập các thành phần nội tại của pipeline (như `classifier`), ta phải tường minh lấy ra pipeline tốt nhất qua thuộc tính `.best_estimator_`.
 - (d) Chỉ cần truy cập `.best_estimator_` trong lần chạy đầu tiên.
7. Trong Gradient Boosting, cơ chế “sửa sai” hoạt động bằng cách nào, và “sai số” được mô hình kế tiếp học dựa trên đại lượng gì?
- (a) Mô hình kế tiếp học trên các mẫu bị phân loại sai, với trọng số của các mẫu này được tăng lên.
 - (b) “Sai số” là phần chênh lệch giữa nhãn thực tế và dự đoán hiện tại, mô hình kế tiếp được huấn luyện để dự đoán chính các phần dư này.
 - (c) “Sai số” được tính bằng gradient bậc hai của hàm mất mát và dùng để cập nhật các nút lá.
 - (d) Mỗi mô hình mới được huấn luyện trên một mẫu bootstrap, và “sai số” được tổng hợp bằng cách lấy trung bình.
8. Đây là sự khác biệt lý thuyết cốt lõi trong cơ chế “sửa sai” tuần tự giữa AdaBoost và Gradient Boosting được mô tả trong tài liệu?
- (a) Cả hai thuật toán đều tính toán phần dư (residual), nhưng AdaBoost chỉ áp dụng cho bài toán phân loại nhị phân.
 - (b) AdaBoost điều chỉnh trọng số của các mẫu huấn luyện, buộc mô hình sau tập trung vào các mẫu bị phân loại sai; trong khi Gradient Boosting huấn luyện mô hình sau để trực tiếp dự đoán phần dư của mô hình tổng hợp trước đó.
 - (c) AdaBoost luôn sử dụng các cây quyết định rất nông (stumps) làm mô hình yếu, trong khi Gradient Boosting phải sử dụng cây sâu hơn để tính toán phần dư.
 - (d) AdaBoost sử dụng learning rate để kết hợp các mô hình yếu, trong khi Gradient Boosting thì không.
9. Việc dùng `fit_transform` cho tập train nhưng chỉ `transform` cho tập val/test trong đoạn code sau nhằm tránh vấn đề cốt lõi nào?

```

1 X_raw_train = raw_pipeline.fit_transform(X_train, y_train)
2 X_raw_val = raw_pipeline.transform(X_val)

```

```
3 X_raw_test = raw_pipeline.transform(X_test)
4
```

- (a) Lỗi tương thích phiên bản của Scikit-learn.
 - (b) Giảm thời gian tính toán vì **transform** nhanh hơn.
 - (c) Rò rỉ dữ liệu, bằng cách đảm bảo các tham số (ví dụ: trung vị, tham số scaling) chỉ được học từ tập huấn luyện.
 - (d) Ngăn ngừa overfitting bằng cách giảm số đặc trưng của tập val/test.
10. Lý do chính đằng sau việc tạo ra đặc trưng tương tác như **chol/age** là gì?

```
1 if {'chol', 'age'} <= set(df.columns):
2     df['chol_per_age'] = df['chol']/df['age']
3
```

- (a) Để chuẩn hóa giá trị cholesterol về cùng thang đo với tuổi.
- (b) Giả định rằng tác động của cholesterol lên nguy cơ bệnh tim có thể thay đổi theo độ tuổi (ví dụ, cholesterol cao ở người trẻ sẽ nguy hiểm hơn).
- (c) Đây là kỹ thuật bắt buộc để xử lý giá trị ngoại lai trong cột **chol** và **age**.
- (d) Để giảm số chiều dữ liệu bằng cách kết hợp hai đặc trưng.

IV. Tài liệu tham khảo

- [1] A. Janosi et al., *Heart disease*, UCI Machine Learning Repository, 1989. DOI: 10.24432/C52P4X. [Online]. Available: <https://doi.org/10.24432/C52P4X>.
- [2] R. E. Schapire, “A brief introduction to boosting”, in *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence (IJCAI-99)*, Stockholm, Sweden: Morgan Kaufmann, 1999, pp. 1401–1406.
- [3] T. Chen et al., “XGBoost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA: ACM, Aug. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Hint:** Các file code gợi ý có thể được tải tại [đây](#).
3. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại [đây](#).
4. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại [đây](#).
5. **Rubric:**

Mục	Kiến Thức	Đánh Giá
I.	- Hiểu bối cảnh bệnh tim mạch và tầm quan trọng của chẩn đoán sớm. - Nắm được thông tin về bộ dữ liệu Cleveland Heart Disease (303 mẫu, 13 đặc trưng). - Biết các đặc trưng y tế chính (age, sex, cp, trestbps, chol, thalach, oldpeak, slope, ca, thal, ...).	- Trình bày được lý do chọn bộ dữ liệu Cleveland và cách nhĩ phân hoá nhãn mục tiêu. - Nhận biết và giải thích ý nghĩa các đặc trưng trong dữ liệu. - Hiểu mối liên hệ giữa dữ liệu y tế và mô hình học máy; nêu được cảnh báo phi y khoa.
II.	- Kiến thức về chọn đặc trưng bằng Decision Tree importance và Mutual Information. - Hiểu cấu trúc 4 biến thể dữ liệu: Original, FE, Original+DT(Top-K), FE+DT(Top-K).	- Giải thích vì sao chọn K đặc trưng và tác động đến bias/variance. - Phân biệt cách xử lý biến phân loại (giữ nguyên, OHE, scaling). - Trình bày đúng pipeline fit/transform cho train/val/test, tránh data leakage.
III.	- Kiến thức về các mô hình ensemble nâng cao: Random Forest, AdaBoost, Gradient Boosting, XGBoost. - Kiến thức về baseline Deep Learning (MLP) trên dữ liệu bảng nhỏ. - Hiểu sự khác biệt Bagging vs Boosting và cơ chế học sửa sai.	- Giải thích cơ chế của RF (giảm phương sai) và Boosting (giảm bias). - Nêu nguyên lý trọng số lỗi trong AdaBoost, residual trong Gradient Boosting. - Trình bày ưu/nhược điểm của XGBoost; khi nào nên dùng GPU/hist. - Nhận diện khi nào MLP hữu ích hoặc dễ overfit trên bộ nhỏ.
IV.	- Kiến thức triển khai bằng scikit-learn/xgboost: pipeline, GridSearchCV, cross-validation. - Biết sử dụng StratifiedKFold và chia train/val/test cố định để so sánh công bằng. - Biết đánh giá bằng Accuracy, Precision, Recall, F1 (ROC-AUC nếu có), và trực quan hoá.	- Viết và hiểu các hàm <code>evaluate_val_*</code>, <code>evaluate_test_*</code>. - Giải thích mục đích của StratifiedKFold trong tìm tham số tối ưu. - Đọc và phân tích được chênh lệch val/test (overfit/underfit). - So sánh hiệu quả giữa các biến thể dữ liệu và mô hình khác nhau.

V.	- Kiến thức về tái lập thí nghiệm: cố định seed, quản lý dữ liệu nhất quán. - Nhận diện và tránh data leakage. - Biết tổ chức code: module hóa, ghi log, tách rõ phần build dữ liệu và train mô hình.	- Cho thấy chạy lại với cùng seed cho kết quả tương đương. - Chỉ ra các điểm dễ gây rò rỉ dữ liệu và cách khắc phục. - Code rõ ràng, có chú thích; quản lý đường dẫn/thư mục đúng chuẩn.
VI.	- Kiến thức về xây dựng giao diện với Streamlit. - Biết cách deploy trên Colab thông qua cloudflared. - Biết đóng gói pipeline (preprocessing + model) để dự đoán trực tiếp từ input.	- Ứng dụng chạy được: nhập tham số, trả dự đoán và xác suất. - Vẽ cây quyết định từ best estimator; hiển thị kết quả dễ hiểu. - Xử lý đúng vấn đề version (sklearn, OHE sparse_output).

6. **Đề xuất phương án cải tiến project:** Trong quá trình triển khai, project đã xây dựng được pipeline hoàn chỉnh từ xử lý dữ liệu, huấn luyện mô hình cho đến ứng dụng demo. Tuy nhiên, vẫn có nhiều hướng mở rộng và tinh chỉnh để nâng cao chất lượng kết quả cũng như khả năng ứng dụng. Các gợi ý dưới đây tập trung vào bốn khía cạnh chính: dữ liệu, mô hình, quản lý thí nghiệm và sản phẩm demo.

- **Cải thiện dữ liệu:**

- Thử nghiệm nhiều phương pháp chuẩn hoá khác nhau như: StandardScaler, MinMaxScaler, RobustScaler..., và so sánh ảnh hưởng đến hiệu suất.
- Sử dụng `VarianceThreshold` để loại bỏ đặc trưng ít biến thiên; áp dụng ma trận tương quan để loại bỏ đặc trưng trùng lặp.
- Tạo thêm đặc trưng mới từ dữ liệu sẵn có (feature engineering) tương tự như một cách làm ví dụ được trình bày ở nội dung bài viết phần 1.
- Ứng dụng `SelectKBest` hoặc `PolynomialFeatures` để khai thác đặc trưng bậc cao.

- **Đa dạng hóa mô hình:**

- Bổ sung Logistic Regression để so sánh với các mô hình đã triển khai.
- Thử nhiều giá trị k trong KNN hoặc sử dụng weighted KNN để đánh giá độ ổn định.
- Áp dụng Support Vector Machine (SVM) với kernel tuyến tính hoặc RBF để kiểm chứng khả năng phân tách dữ liệu.

- **Quản lý thí nghiệm:**

- Đặt seed cố định để đảm bảo tính tái lập kết quả.
- Ghi chú và lưu lại cấu hình, tham số, kết quả nhằm thuận tiện cho việc so sánh.
- Sử dụng Optuna hoặc GridSearchCV mở rộng để tự động hóa quá trình tối ưu tham số.

- **Cải thiện sản phẩm demo:**

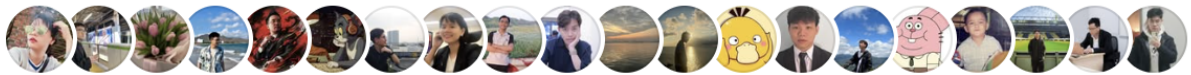
- Thiết kế giao diện nhập liệu trực quan hơn: dùng `st.slider` cho biến số (tuổi, cholesterol) và `st.selectbox` cho biến phân loại (giới tính, loại đau ngực).

- Hiển thị xác suất dự đoán bên cạnh nhãn phân loại để làm rõ mức độ rủi ro.
- Bổ sung phần giải thích đặc trưng quan trọng (feature importance) từ mô hình.
- Lưu lại lịch sử dự đoán trong ứng dụng hoặc triển khai bằng Docker/Streamlit Cloud để tăng tính khả chuyển.

7. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_Family

🔒 Private group · 2.0K members



Hình 16: Hình ảnh group facebook AIVN Family.